



INGENIERÍA

Navegación Autónoma de Drones Urbanos con Visión Monocular y Small Language Model (SLM)

Tesis para Magister en Ciencia de Datos

Maestría en Ciencia de Datos - 2024/2025

Directores:

- [DEL ROSSO, Rodrigo](#)
- [NUSKE, Ezequiel](#)

Alumno:

- [NICOLAU, Jorge Enrique](#)

Resumen

La navegación autónoma de cuadricópteros eVTOL en entornos urbanos densos —como el de Buenos Aires— enfrenta severos desafíos derivados de la alta densidad de obstáculos, la degradación de señales satelitales y las restricciones presupuestarias que descartan sensores de alto costo como LiDAR. Para abordar esta problemática bajo estrictas restricciones de hardware y en conformidad con las normativas locales (ANAC Res. 880/2019), esta tesis presenta un sistema integral de planificación previa, percepción monocular y decisión táctica basado exclusivamente en **visión monocular** y **modelos de lenguaje pequeños (SLM)** locales.

La arquitectura desacopla un módulo de planificación deliberativa en tierra (estación de control WebDCS, que compila directivas en lenguaje natural a un manifiesto inmutable de navegación) de un lazo táctico a bordo de alta frecuencia. El módulo de percepción implementa un pipeline ligero de **flujo óptico denso derotado** mediante telemetría de actitud, mapeo de ocupación espacial (`ObstacleField`) y estimación de **Tiempo-a-Colisión (TTC)** a partir de la divergencia del campo de flujo, alcanzando un área bajo la curva ROC de 0.96–0.97 en su validación frente a canales de profundidad de referencia. En la capa de decisión a bordo, se implementa una arquitectura híbrida de decisión por ciclo gobernada por un SLM (modelos de 2B–4B parámetros ejecutados localmente con decodificación estructurada `json_schema`) y orquestada mediante grafos de estados con salvaguardas deterministas.

El desarrollo y la experimentación se realizan en el simulador **Cosys-AirSim** sobre **Unreal Engine 5.5**, empleando el entorno urbano **CitySim** como plataforma principal de validación y benchmark, y calibrando la dinámica de vuelo contra conjuntos de datos de telemetría de drones comerciales reales. El trabajo documenta una evaluación comparativa sistemática del brazo SLM frente a Máquinas de Estados Finitos (FSM) y control puramente reactivo, analizando métricas de tasa de éxito de misión, tiempo de reacción y longitud de trayectoria ponderada por éxito (SPL). Asimismo, se identifican y analizan los modos de falla estructurales propios de sistemas robóticos asistidos por modelos de lenguaje, demostrando que los errores críticos en la toma de decisiones se originan predominantemente en degradaciones silenciosas del contrato de datos de la interfaz de percepción antes que en el razonamiento del modelo.

Palabras clave

Navegación Autónoma de Drones | Visión Monocular | Flujo Óptico | Tiempo-a-Colisión (TTC) | Small Language Models (SLM) | Máquinas de Estados Finitos (FSM) | Control Táctico Híbrido | Simulación en Unreal Engine / Cosys-AirSim | Robótica Urbana de Bajo Costo

Table of Contents

- 1 Carátula y Resumen
- 2 1. Introducción
- 3 2. Estado del Arte
- 4 3. Entorno de Simulación
- 5 4. Planificación de Misión (WebDCS)
- 6 5. Arquitectura del Lazo Táctico
- 7 6. Percepción Monocular
- 8 7. Estimación TTC
- 9 8. Decisiones SLM
- 10 9. Modos de Falla LLM
- 11 10. Metodología Experimental
- 12 11. Resultados
- 13 12. Conclusiones
- 14 13. Referencias
- 15 Anexos

1. Introducción

1.1 Motivación y contexto

La navegación autónoma de drones en entornos urbanos densos —como el de Buenos Aires— enfrenta un conjunto de restricciones que no aparecen en espacios abiertos: alta densidad de obstáculos fijos y móviles, degradación de la señal GPS por la altura de los edificios, condiciones de iluminación variables y, en el contexto latinoamericano en particular, escasez de conjuntos de datos locales y restricciones de presupuesto que descartan sensores costosos como el LiDAR. Un dron que carezca de percepción confiable y de una capacidad de decisión robusta representa un riesgo tangible: puede colisionar, invadir zonas sensibles o interferir con otras operaciones, con consecuencias humanas y económicas concretas (Samy et al., 2019; Aldao et al., 2022).

Este trabajo se enmarca en esa tensión: cómo dotar a un cuadricóptero eVTOL de navegación autónoma reactiva usando exclusivamente visión monocular y hardware de bajo costo, cumpliendo además con el marco regulatorio local (Resolución ANAC 880/2019), sin resignar seguridad ni rigor metodológico. La motivación original —detallada en el plan de trabajo aprobado (`plan_tesis/plan-tesis.md`)— incluye aplicaciones de logística de última milla, inspección de infraestructura y gestión de emergencias urbanas, todas ellas dominadas por el mismo problema técnico de fondo: decidir, en tiempo real y con percepción limitada, si el camino hacia adelante está despejado.

1.2 Objetivos

Objetivo general. Desarrollar un sistema de visión por computadora basado en visión monocular con capacidad de navegación autónoma de cuadricópteros eVTOL, aplicable a logística urbana, mantenimiento de infraestructura y atención de emergencias en entornos como el de Buenos Aires.

Objetivos específicos:

1. Implementar un pipeline reproducible sobre el simulador AirSim (basado en Unreal Engine) que permita validar el desempeño del sistema de navegación en condiciones controladas y repetibles.
2. Integrar detección de obstáculos y navegación reactiva a partir de percepción monocular ligera, optimizada para hardware de bajo costo, de modo de sostener un caso de negocio viable para economías con recursos limitados.
3. Evaluar el rendimiento de un modelo de lenguaje pequeño (SLM) como capa de decisión táctica frente a una máquina de estados finitos (FSM) —el estándar de facto en pilotos automáticos—, usando tasa de éxito de misión, tiempo de reacción y consumo computacional como métricas de comparación.

1.3 Alcance y diseño de la arquitectura

El sistema de navegación propuesto implementa una arquitectura de percepción monocular ligera **sin redes neuronales de detección**, basada en flujo óptico denso derotado mediante telemetría de actitud y estimación física de tiempo-a-colisión (TTC). Esta elección de diseño responde a fundamentos técnicos y computacionales concretos:

1. **Eficiencia y presupuesto de cómputo:** En plataformas robóticas de bajo costo que comparten GPU o CPU con la inferencia de un modelo de lenguaje local, la estimación geométrica por flujo óptico reduce drásticamente la latencia por ciclo frente a detectores de objetos basados en aprendizaje profundo.
2. **Robustez fuera de distribución (OOD):** A diferencia de modelos supervisados dependientes de categorías predefinidas de obstáculos (vehículos, peatones, postes), el cálculo de divergencia del campo de flujo modela directamente el fenómeno físico de aproximación, reaccionando ante cualquier geometría sin importar su textura o clase semántica.
3. **Adecuación a la geometría del vuelo urbano:** Para un cuadricóptero con cámara frontal en cañón urbano, el campo visual está dominado por estructuras verticales y fachadas; el flujo óptico traslacional permite estimar la ocupación espacial (`ObstacleField`) en cuadrantes discretos de forma directa y determinista.

El capítulo 6 desarrolla en detalle los fundamentos matemáticos y la implementación de este módulo de percepción.

1.4 El hilo conductor de la tesis

En arquitecturas robóticas híbridas donde un modelo de lenguaje consume descripciones simbólicas o estructuradas del entorno para tomar decisiones de navegación, la integridad del contrato de interfaz entre la percepción y el modelo resulta crítica. La experiencia experimental demuestra que **el fallo más severo en estos sistemas no radica en la capacidad de razonamiento del modelo de lenguaje, sino en la fidelidad y consistencia del contrato de datos que lo alimenta**.

Cuando la capa de percepción o el estado del sistema entrega representaciones desincronizadas, degradadas o vacías, los modelos de lenguaje tienden a generar respuestas sintácticamente válidas y aparentemente razonables a partir de premisas falsas. Dado que el modelo no se interrumpe con excepciones visibles y el vehículo continúa su trayectoria, estas fallas resultan estructuralmente invisibles si solo se observa el comportamiento cinemático superficial.

Este principio —la necesidad de validación formal, auditoría de contratos de datos y salvaguardas deterministas en lazos de control asistidos por SLM— constituye el hilo conductor de este trabajo. El capítulo 9 documenta de manera sistemática los modos de falla estructurales identificados en la integración de estos componentes y las salvaguardas implementadas para mitigarlos, retomándose como conclusión central en el capítulo 12.

1.5 Estructura del informe

El capítulo 2 sitúa este trabajo respecto de la literatura sobre navegación autónoma de UAVs, percepción monocular para detección de obstáculos y arquitecturas de control asistidas por modelos de lenguaje. El capítulo 3 describe la construcción del entorno de simulación —Unreal Engine 5.5 con Cosys-AirSim y los entornos urbanos empleados— y su validación estadística frente a telemetría de vuelos reales. El capítulo 4 documenta la planificación deliberativa en tierra y la estación de control WebDCS. Los capítulos 5 a 9 documentan la arquitectura del lazo táctico a bordo: el lazo táctico de control (cap. 5), la percepción monocular sin redes neuronales (cap. 6), la estimación y validación del TTC (cap. 7), la capa de decisión del SLM (cap. 8) y los modos de falla específicos de lazos de control híbridos con modelos de lenguaje (cap. 9). El capítulo 10 describe la metodología experimental —diseño, brazos de comparación y métricas— con la que se evalúa el sistema. Los capítulos 11 y 12 (resultados comparativos y conclusiones) sintetizan los hallazgos y el trabajo futuro, complementados por las referencias bibliográficas estructuradas en el capítulo 13.

2. Estado del arte y trabajos relacionados

2.1 Navegación autónoma de UAVs: de la planificación clásica al control adaptativo

Los primeros enfoques de navegación autónoma en entornos urbanos combinan planificación global sobre datos geográficos con evasión reactiva de obstáculos móviles. Castelli et al. (2016) proponen un sistema híbrido para vuelo seguro a baja altitud que combina el algoritmo *A con datos GIS y replanificación dinámica, priorizando rutas sobre techos en lugar de calles para aumentar la distancia de seguridad frente a objetos en movimiento*. Hughes y Engelbrecht (2023) extienden esta idea a enjambres, con planificación de largo plazo y evasión cooperativa basada en bounding volume hierarchies* y diagramas de Voronoi sobre un modelo 3D tipo Manhattan —el mismo tipo de trazado urbano en cuadrícula que emplea el entorno de evaluación densa de este trabajo (CitySim / Tier 2; ver capítulos 3 y 10)—.

Frente a estos enfoques deliberativos, las máquinas de estados finitos (FSM) siguen siendo el estándar de facto en pilotos automáticos por su predictibilidad y bajo costo computacional: Hu et al. (2024) las emplean para coordinación cooperativa de enjambres en emergencias, y Hoang et al. (2024) las utilizan como base de un sistema de recarga autónoma sobre líneas de alta tensión. Su limitación es estructural: una FSM no interpreta contexto más allá de los umbrales sobre los que fue diseñada. Ese contraste —predictibilidad y bajo costo de la FSM frente a la promesa de adaptabilidad contextual de un modelo de lenguaje— es precisamente el eje de la comparación experimental de este trabajo (cap.10), y es también donde AlMahamid y Grolinger (2022) y Bouhamed et al. (2020) sitúan al aprendizaje por refuerzo como tercera vía: políticas aprendidas en lugar de reglas explícitas o inferencia de un modelo de lenguaje.

2.2 Percepción monocular para detección y evitación de obstáculos

La adopción de una arquitectura de percepción monocular ligera sin redes neuronales profundas (cap. 6) se fundamenta en una línea de investigación consolidada sobre detección de obstáculos por visión artificial clásica. Badrloo y Varshosaz (2017) revisan el estado del arte de la detección monocular en robótica móvil, mientras que Molineros et al. (2012) y Chen et al. (2016) analizan la estimación de riesgo mediante flujo residual y análisis 3D.

Por otra parte, técnicas clásicas basadas en Mapeo de Perspectiva Inversa (IPM), como las propuestas por Kaneko et al. (2017) y retomadas por Zhou et al. (2026), asumen la existencia de un plano de suelo dominante en el campo de visión, una hipótesis adecuada para vehículos terrestres o cámaras cenitales pero inaplicable a cuadricópteros con cámara frontal a baja altitud en cañones urbanos, donde el campo visual está dominado por estructuras verticales y fachadas.

En contraste, el uso de flujo óptico denso traslacional constituye una solución físicamente rigurosa y libre de entrenamiento supervisado. Vera-Yanez et al. (2024) desarrollan un detector de obstáculos aéreos basado íntegramente en flujo óptico —morfología, foco de expansión (*focus of expansion*, FOE) y agrupamiento—, demostrando que desacoplar la rotación propia del movimiento traslacional permite aislar los vectores de aproximación generados por objetos en la trayectoria. Este principio físico —la divergencia del campo traslacional como estimador directo del Tiempo-a-Colisión (TTC)— constituye la base matemática del módulo de percepción y estimación de TTC implementado en este trabajo (cap. 6 y 7). Enfoques alternativos basados en aprendizaje profundo monocular (Rill y Faragó, 2021; Shi et al., 2024) logran estimaciones densas de profundidad pero introducen una alta carga computacional y sensibilidad a dominios no vistos, justificando la preferencia por métodos geométricos deterministas en plataformas de bajo costo.

2.3 Arquitecturas de control asistidas por modelos de lenguaje

La incorporación de modelos de lenguaje al lazo de control de UAVs se ha vuelto, hacia 2025-2026, un patrón relativamente estandarizado: un ciclo cerrado en el que el estado del vehículo se traduce a lenguaje natural, un modelo (grande o pequeño) produce una decisión estructurada, y esa decisión se ejecuta y se vuelve a observar. Chahine et al. (2023) muestran robustez fuera de distribución con redes neuronales líquidas como capa de control; Zhu et al. (2024) estudian específicamente hasta qué punto los modelos de lenguaje entienden el contexto que se les provee, una pregunta directamente relevante para el capítulo 9 de este trabajo, donde el problema no es la capacidad de razonamiento del modelo sino la integridad del contexto que efectivamente recibe.

Dos líneas de trabajo son especialmente pertinentes para la ingeniería de decisiones del SLM descrita en el capítulo 8. Por un lado, la generación de salida estructurada y restringida: Geng et al. (2025) sistematizan el estado del arte en generación de salidas estructuradas desde modelos de lenguaje —el problema exacto que resuelve, en este trabajo, la decodificación restringida vía `json_schema` con parser tolerante como red de contención (cap. 8)—, y Raspanti et al. (2025) muestran que la decodificación con gramática restringida mejora específicamente el desempeño en tareas de análisis lógico estructurado. Por otro lado, la simulación de alta fidelidad como banco de pruebas: Shah et al. (2017) introducen AirSim, el simulador sobre el que se desarrolla y valida este trabajo, y Jansen et al. (2023) presentan una variante extendida (Cosys-AirSim) para aplicaciones industriales complejas, evidencia de que AirSim sigue siendo, varios años después, una base activa para este tipo de investigación.

2.4 Posicionamiento de este trabajo

Este trabajo se ubica en la intersección de esas tres líneas —percepción monocular sin redes neuronales, control por FSM como línea de base, y un SLM con salida restringida como capa de decisión alternativa— pero con un énfasis distinto al de buena parte de la literatura revisada: en lugar de proponer una arquitectura novedosa de percepción o de razonamiento, documenta con evidencia medida los modos de falla específicos de integrar un modelo de lenguaje en un lazo de control real, un tipo de resultado que —según constata el propio capítulo 9— es poco frecuente en la literatura publicada, que tiende a reportar arquitecturas que funcionan más que arquitecturas que fallaron de una manera instructiva y cómo se detectó y corrigió esa falla.

3. Construcción y validación del entorno de simulación

3.1 Arquitectura de simulación: Unreal Engine 5.5 y Cosys-AirSim

El desarrollo y la validación de este trabajo se realizan, tal como preveía el plan de trabajo aprobado (`plan_tesis/plan-tesis.md`), sobre AirSim, un simulador basado en Unreal Engine que ofrece física y renderizado de alta fidelidad (Shah et al., 2017). La implementación efectiva, sin embargo, no usa la distribución original de AirSim de Microsoft: desde el inicio del proyecto se adoptó **Cosys-AirSim**, un fork mantenido por el Cosys-Lab (Laboratorio de Co-Diseño para Sistemas Ciber-Físicos de la Universidad de Amberes, Bélgica). Al inicio del desarrollo se adoptó la decisión de utilizar este fork: *"Abandonado el proyecto original AirSim por Microsoft, se utiliza la actual versión a partir de un fork mantenido por el Cosys-Lab"*, compilado e integrado sobre proyectos de Unreal Engine 5.5 que sirvieron como banco de pruebas y validación experimental.

Cosys-AirSim, a diferencia del AirSim clásico de Microsoft —enfocado principalmente en cámaras RGB y visión por computadora—, agrega sensores adicionales basados en GPU y CPU (LiDAR, sonar, radar), documentados en el paper oficial de la plataforma (Jansen et al., 2023). Este trabajo no usa esos sensores adicionales: la percepción descrita en el capítulo 6 depende exclusivamente de la cámara RGB monocular y de la telemetría de actitud, consistente con la restricción de hardware de bajo costo que motiva todo el proyecto (§1.2). La elección de Cosys-AirSim sobre el AirSim original responde, de todos modos, a la actividad de mantenimiento del fork más que a una necesidad de esos sensores adicionales: hacia 2025-2026 el proyecto de Microsoft dejó de recibir actualizaciones activas, y la documentación, configuración y versiones del fork de Cosys-Lab fueron revisadas exhaustivamente antes de adoptarlo.

Particularidades de la API y telemetría de actitud. Una diferencia relevante descubierta durante la integración radica en la interfaz en Python: a diferencia del paquete clásico `airsim`, el binding `cosysairsim` no expone la función auxiliar `to_eulerian_angles` para la extracción directa de ángulos de actitud. La ausencia de este método provocó que los fallbacks iniciales reportaran *pitch* y *roll* en 0.0° de forma permanente. Para subsanar esta omisión sin introducir dependencias externas complejas, se implementó en el cliente de simulación (`AirSimClient._quaternion_to_euler`) la conversión matemática directa de cuaterniones a ángulos de Euler en convención aeroespacial NED (North-East-Down), asegurando que las variaciones de inclinación del cuadricóptero alimenten con fidelidad continua la telemetría del lazo táctico y el contexto del modelo deliberativo.

3.2 Desvío respecto del plan aprobado: de la fotogrametría de Buenos Aires a activos urbanos genéricos

El plan de trabajo aprobado describía un pipeline de digitalización de entornos urbanos específico: extraer fotogramas de videos públicos de drones en Buenos Aires (YouTube), filtrarlos para asegurar diversidad visual, anotar obstáculos y zonas de aterrizaje con LabelImg/LabelMe, y reconstruir modelos 3D hiperrealistas con RealityCapture (fotogrametría) a partir de esos videos y de OpenStreetMap, optimizados en Blender para reducir su complejidad computacional antes de importarlos a AirSim (`plan_tesis/plan-tesis.md`, §"Metodología"). El objetivo declarado era un entorno virtual 3D *personalizado* de Buenos Aires, con las particularidades geométricas de su infraestructura densa.

La implementación efectivamente construida sustituyó ese pipeline de fotogrametría por activos urbanos y suburbanos ya disponibles para Unreal Engine (marketplace de Fab/Epic Games y paquetes de terceros), configurados con el plugin de Cosys-AirSim en lugar de reconstruidos a partir de video real de Buenos Aires (disponibles en el repositorio compartido de Google Drive:

<https://drive.google.com/drive/folders/1roLmbGFNsHXZyT3NaNzNYMuaBQ8CulX7>).

Entornos principales estructurados por Tiers de evaluación

Para responder con rigor al protocolo experimental de la tesis (capítulo 10), los entornos de simulación se organizaron en una **jerarquía formal de tres niveles de complejidad (Tiers)**, superando los primeros borradores preliminares de misión (`manhattan_a` y `manhattan_b`, descartados y alineados a esta nueva nomenclatura):

1. **Tier 0 — Base / Control (MiniSim)**: Entorno abierto basado en el mapa `crater.png` (`Landscape Mountains`). Diseñado para pruebas de calibración cinemática, sintonización de controladores de posición y determinación de la cota inferior de latencia del sistema sin interferencia de obstáculos.
2. **Tier 1 — Intermedio / Morfología orgánica y vegetación (TownSim)**: Entorno semiurbano desarrollado sobre `townsim.png` y refinado en el mapa calibrado `townsim_calib.png`. Presenta calles de ancho variable, fachadas intermedias, mobiliario y arboledas densas. Constituye el banco de pruebas central para la batería de calibración `T-CALIB-0` a `T-CALIB-5` y la misión de recorrido perimetral `townsim_ini`, albergando el escenario de bloqueo frontal masivo genuino (`T-CALIB-2`).
3. **Tier 2 — Complejo / Cañones urbanos de gran densidad (CitySim / CityParkSim)**: Proyecto de Unreal Engine 5.5 con tejido edilicio masivo (`citysim_calib.png`), rascacielos, pasos restringidos y disposición en cuadrícula ortogonal. El escenario base validado para el batch G4 es `citysim_clear.json`: perímetro de manzana con patrón *climb-first*, ascenso vertical puro hasta -70 m AGL antes de moverse horizontalmente. La altitud de tránsito de -70 m es un dato de diseño del entorno: la primera corrida a -50 m falló porque la ruta de ascenso atravesaba edificios; a -70 m el dron vuela por encima de la línea de tejados. Los escenarios con corredores angostos (`citymap_pilot.json` y `citymap_a.json`) corresponden a la fase siguiente del batch.

Entornos adicionales evaluados en fases exploratorias

Junto a los tres Tiers principales, se evaluaron otras alternativas durante el desarrollo: - **"City Sample"** (Epic Games, vía Fab): entorno dedicadamente urbano denso, con peatones y tráfico gestionado por IA autónoma de Unreal Engine, confirmado funcionando con Cosys-AirSim el 2026-0522 y optimizado el 2026-0622 (escena `Small_City_LVL`) siguiendo las recomendaciones oficiales de Epic para sistemas de especificación más baja. - **"Downtown West Modular Pack"**: entorno semiurbano con mayor nivel de detalle arquitectónico, configurado el 2026-0521 como alternativa de mayor realismo visual al entorno base. - **"Dynamic City Creator"** (2026-0509): intento de generar un entorno urbano paramétricamente, **abandonado** debido a que Cosys-AirSim no detectaba adecuadamente la malla de colisión de las geometrías generadas proceduralmente, invalidando la detección física de obstáculos.

El historial del proyecto no deja registrada una justificación explícita de por qué se sustituyó la fotogrametría de Buenos Aires por estos entornos; se documenta aquí como un desvío de hecho respecto del plan aprobado, en el mismo sentido en que el capítulo 1 (§1.3) documenta el desvío en percepción. Una lectura razonable es que los activos preexistentes resolvían de forma inmediata la necesidad de geometrías urbanas complejas con colisiones operativas, ahorrando el enorme costo técnico y temporal del pipeline de reconstrucción fotogramétrica.

3.3 Configuración de rendimiento, escalabilidad gráfica e higiene de captura

La documentación técnica de Cosys-Lab sostiene que priorizar la tasa de actualización de la física por sobre la fidelidad gráfica es una práctica recomendada cuando el objetivo principal no es la fotografía cinemática de la escena. Siguiendo esas directrices y considerando que la misma estación de cómputo aloja en simultáneo el renderizado 3D y la inferencia del modelo de lenguaje local (§8.1), se establecieron los siguientes compromisos de arquitectura:

- Desactivar el ahorro de CPU en segundo plano del editor de Unreal Engine (`Use Less CPU when in Background`), evitando caídas drásticas en el refresco de la física cuando la ventana del simulador pierde el foco.
- Ajustar `ClockSpeed` en `settings.json` para sincronizar el tiempo de simulación y garantizar que el lazo físico se calcule sin pérdidas temporales cuando la carga gráfica fluctúa.
- Preferir binarios empaquetados (*standalone*) sobre la ejecución directa desde el editor, reduciendo la sobrecarga de memoria VRAM asociada a la interfaz de desarrollo.
- El modo `NoDisplay` no pudo utilizarse, dado que la captura de imágenes RGB monocular es estrictamente indispensable para la percepción del dron (capítulo 6).

Escalabilidad gráfica en Unreal Engine 5.5

Para equilibrar la carga de GPU entre Unreal Engine y el servidor SLM/VLM local, se definió un perfil de escalabilidad mínima (*Minimal Scalability Config*) que fija sombras, postprocesamiento e iluminación global en niveles bajos (`Low / Medium`), reservando la VRAM necesaria para los pesos del modelo multimodal.

Sin embargo, en Unreal Engine 5.5 la reducción automática de efectos suele degradar la resolución de los render targets de captura de escena. Para evitar que la cámara frontal pierda nitidez o altere el cálculo de flujo óptico, se forzó el nivel de detalle en el archivo `Config/DefaultScalability.ini` del proyecto Unreal:

```
[EffectsQuality@0]
r.DetailMode=2
[EffectsQuality@1]
r.DetailMode=2
[EffectsQuality@2]
r.DetailMode=2
[EffectsQuality@3]
r.DetailMode=2
[EffectsQuality@Cine]
r.DetailMode=2
```

Scalability Groups						
	Low	Medium	High	Epic	Cinematic	Auto
View Distance	Near	Medium	Far	Epic	Cinematic	
Anti-Aliasing (None)	Low	Medium	High	Epic	Cinematic	
Post Processing	Low	Medium	High	Epic	Cinematic	
Shadows	Low	Medium	High	Epic	Cinematic	
Global Illumination	Low	Medium	High	Epic	Cinematic	
Reflections	Low	Medium	High	Epic	Cinematic	
Textures	Low	Medium	High	Epic	Cinematic	
Effects	Low	Medium	High	Epic	Cinematic	
Foliage	Low	Medium	High	Epic	Cinematic	
Shading	Low	Medium	High	Epic	Cinematic	
Landscape	Low	Medium	High	Epic	Cinematic	
Monitor Editor Performance?						

Higiene y consistencia de la captura monocular

Durante la experimentación continua en simulación se identificaron y resolvieron dos anomalías instrumentales críticas en el canal visual:

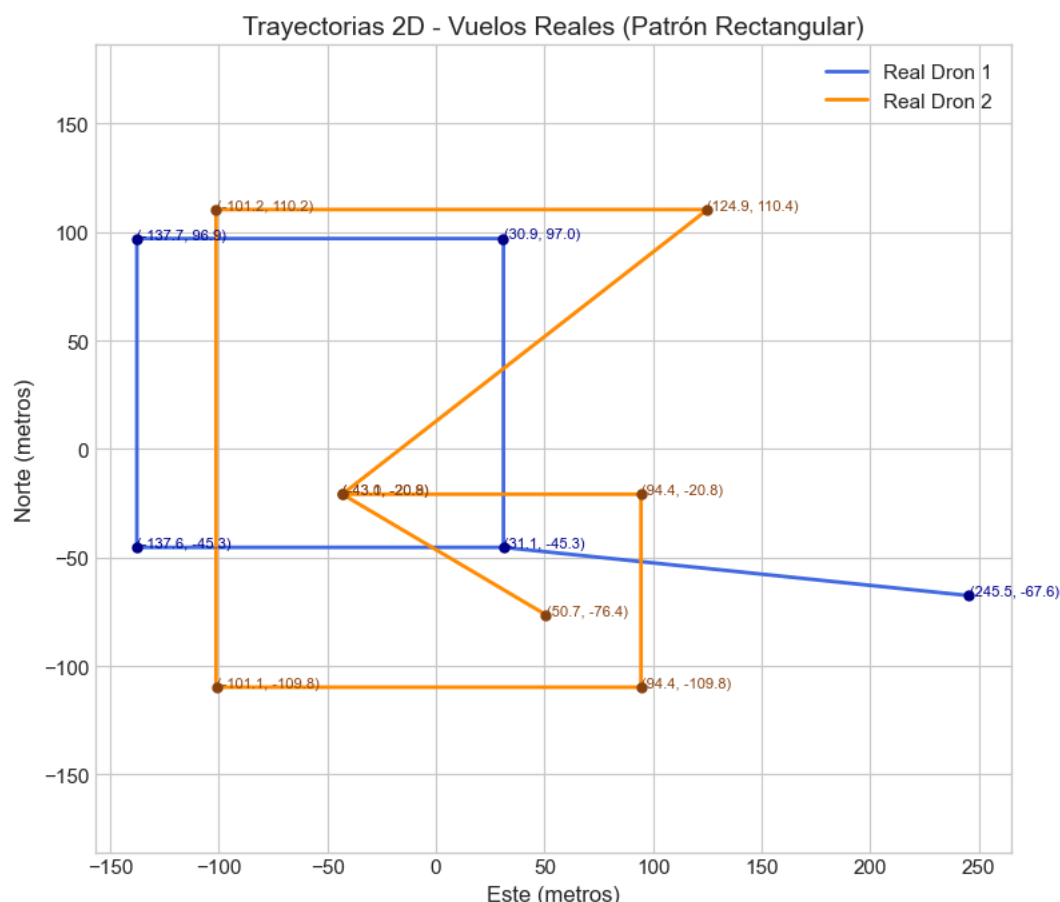
1. **Inversión de canales de color (RGB vs. BGR):** El buffer crudo devuelto por `simGetImages(ImageType.Scene)` en Cosys-AirSim es entregado en formato RGB. No obstante, el ecosistema de procesamiento de OpenCV y las rutinas de compresión asumían internamente la convención BGR. Como consecuencia, las capturas enviadas al VLM y guardadas en auditoría mostraban una tonalidad azulada con rojo y azul invertidos. La anomalía se corrigió de forma definitiva en el propio método `AirSimClient.capture()`, asegurando que el modelo de lenguaje razone sobre los colores naturales de la escena (pavimento, vegetación y cielo).
2. **Supresión de marcadores de depuración persistentes:** Primitivas geométricas dibujadas en el mundo virtual por scripts de planificación (mediante `simPlotLineStrip` y `simPlotPoints` de `plot_mission_route.py`) permanecían activas en el nivel y eran capturadas por la cámara a bordo, siendo interpretadas por el VLM como obstáculos físicos reales. Se implementó una rutina de higiene obligatoria (`AirSimClient.clear_debug_markers()`), invocando `simFlushPersistentMarkers()` al inicio de cada sesión de conexión.

3.4 Validación frente a telemetría de vuelos reales

La dinámica de vuelo simulada —no la geometría de la escena urbana, cuya fidelidad fotográfica es el desvío documentado en §3.2— se validó de forma independiente contra telemetría de drones reales, con el objetivo explícito de que la comparación experimental entre brazos (capítulo 10) se apoyara en vuelos simulados cuya cinemática fuera comparable a la de un cuadricóptero real y no solo físicamente plausible en abstracto.

Fuente de datos reales. Se descargó un conjunto de datos de telemetría real de drones cuadricópteros comerciales (DJI) desde el repositorio público de Zenodo (<https://zenodo.org/records/15912415>).

Protocolo de calibración. Sobre esa telemetría real se identificaron dos trayectorias de vuelo distintas —un patrón rectangular simple (Drone 1) y un patrón de cruz combinado con rectángulo (Drone 2)— y se reprodujeron en simulación con `moveOnPath / move` de la API de AirSim, ajustando la simulación a la misma altura y la misma velocidad que los vuelos reales correspondientes. El proceso se iteró varias veces a lo largo del proyecto: una primera generación automatizada de telemetría de 100 vuelos simulados (2026-0413) para poner a punto el script de iteración; una serie de 10 vuelos de calibración con telemetría registrada en archivos CSV individuales (2026-0409); una repetición con trayectorias explícitamente ajustadas a la altura y velocidad de los vuelos reales (2026-0610); y una iteración final (2026-0627/2026-0628) que reemplazó los comandos de movimiento discretos por `moveOnPath` con la trayectoria completa como un único comando, para acercar el perfil de aceleración simulado al de un vuelo real continuo en lugar de una secuencia de tramos independientes.

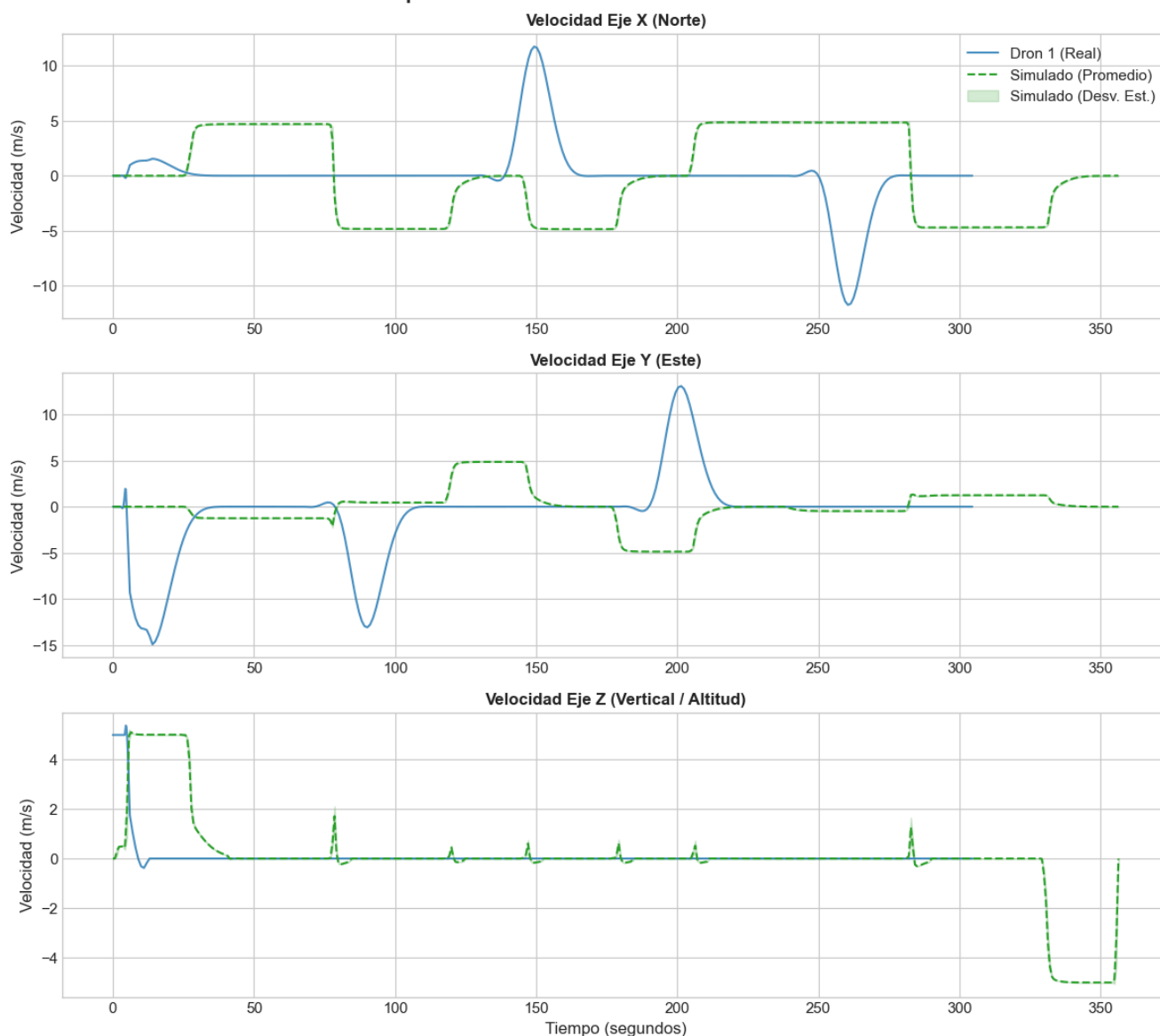


Metodología estadística. El análisis se implementó en notebooks de Jupyter del repositorio complementario del proyecto (`consolidate_telemetry.ipynb` y `telemetry_analysis_*.ipynb`, [georgsmeinung/lm-drone](https://github.com/georgsmeinung/lm-drone)), con estadística descriptiva y pruebas de significancia —en particular la prueba de Levene para diferencia de varianzas— aplicadas al cambio de velocidad en los tres ejes, para determinar si la distribución de la telemetría simulada difiere de forma significativa de la real.

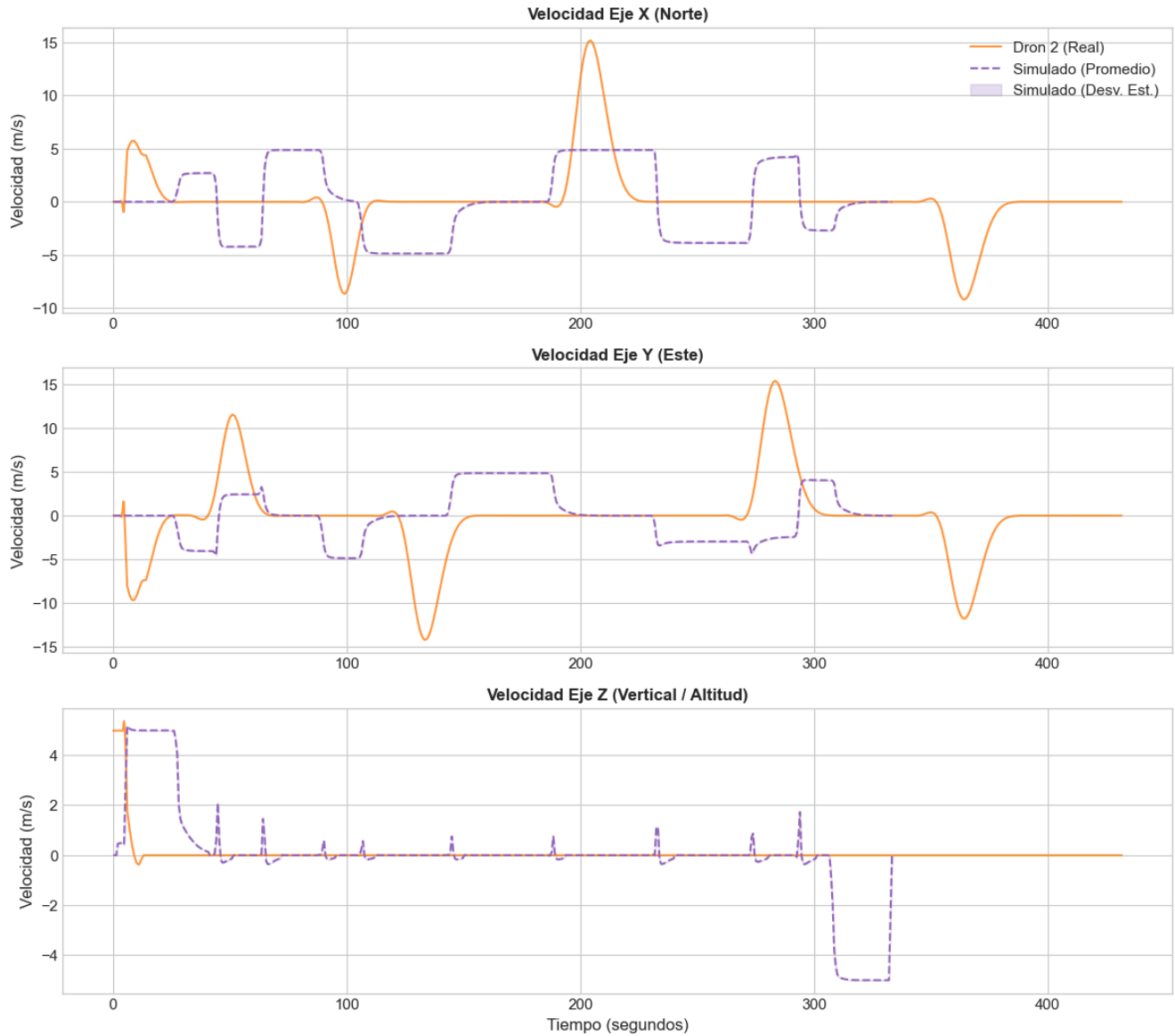
Resultado. Las corridas de análisis (2026-0604 a 2026-0628) documentan tres hallazgos consistentes:

1. **La varianza de actitud difiere de forma significativa** (prueba de Levene, $p \ll 0,05$): en simulación, el dron ejecuta inclinaciones de *roll/pitch* extremas durante los giros rápidos para alcanzar instantáneamente el punto de trayectoria siguiente. Los drones reales, en cambio, están limitados electrónicamente por su controlador PID de estabilización (típicamente a $\pm 30^\circ$), y muestran una varianza mucho menor y acotada durante las mismas maniobras.
2. **La segregación por trayectoria es significativa y consistente.** El Drone 1 (patrón rectangular, giros menos frecuentes) concentra la aceleración en las esquinas; el Drone 2 (patrón de cruz y rectángulo continuo) presenta una dinámica transicional más exigente y oscilaciones de *roll/pitch* más ruidosas, tanto en el vuelo real como en el simulado.
3. **El vuelo simulado en tramos rectos es idealizado.** Durante las fases rectas, la telemetría simulada muestra varianza de actitud cercana a cero, sin fuerzas externas de viento ni ruido de sensores; el dron real, en cambio, mantiene una variabilidad permanente de $\pm 2^\circ$ – 3° en *roll* y *pitch* incluso en tramos rectos estables, producto del viento real y de las correcciones continuas del piloto automático.

Comparación de Perfiles de Velocidad - Dron 1



Comparación de Perfiles de Velocidad - Dron 2



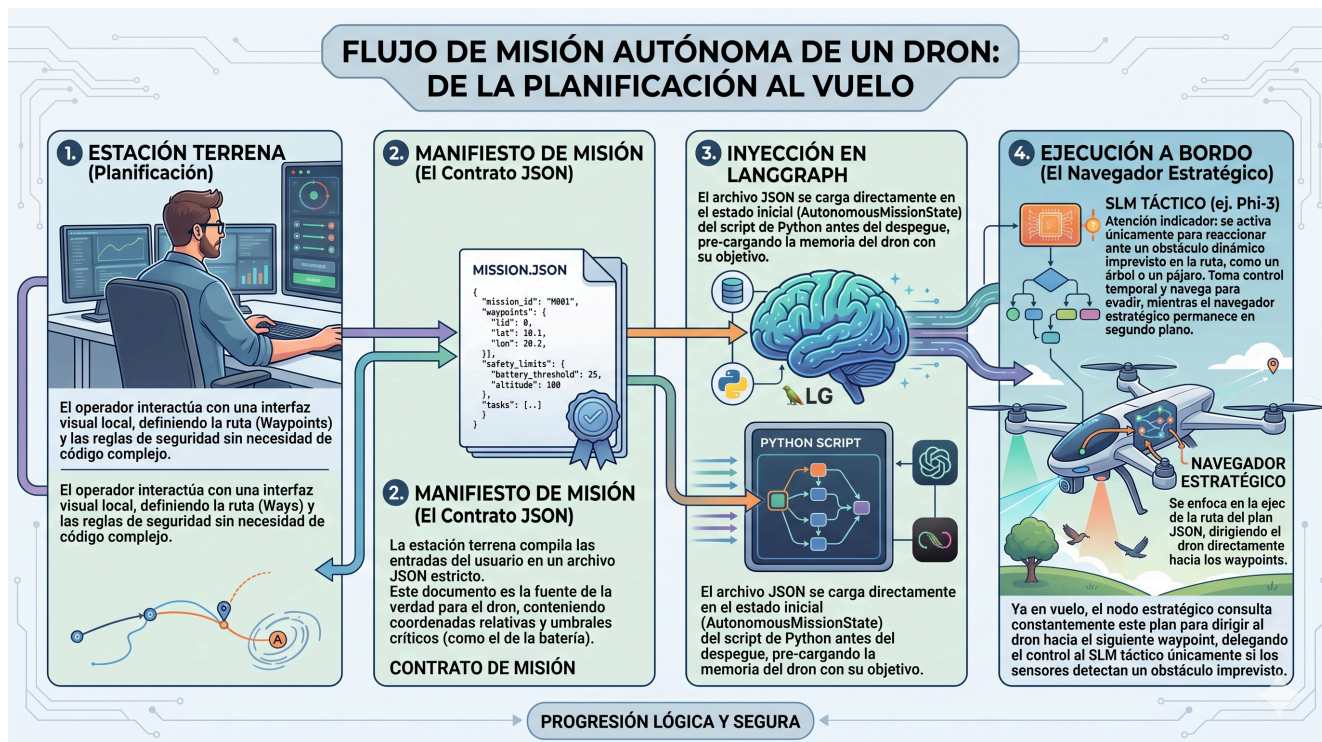
Alcance de esta validación. Confirma que la dinámica de vuelo simulada —cómo se mueve el dron dado un comando— es más ágil y menos amortiguada que la de un dron real equivalente, con una brecha de idealización conocida y cuantificada (varianza de actitud, ausencia de viento simulado). No valida, en cambio, la fidelidad fotográfica de la escena urbana frente a Buenos Aires real: esa es una validación distinta, no realizada, y coincide con el desvío documentado en §3.2. Un ejercicio de validación relacionado pero diferente —si el suavizado por histéresis y media móvil exponencial introducido en `WaypointTracker` (capítulo 5, §5.3) mejora medible la telemetría real de vuelo— arrojó un resultado honestamente ambiguo (`std(Δvx)` prácticamente plano antes/después) y se documenta como tal en el presente informe, no como una mejora confirmada.

4. Planificación de misiones y estación de control en tierra (WebDCS)

4.1 Arquitectura desacoplada: dos cerebros (tierra vs. vuelo)

La arquitectura general del sistema responde a un principio de diseño fundamental: la separación estricta entre la **planificación deliberativa global previa al vuelo** (ejecutada en tierra) y la **navegación táctica reactiva** (ejecutada a bordo durante el vuelo). Esta división, conceptualizada como una arquitectura de *dos cerebros*, resuelve la asimetría intrínseca de recursos computacionales y tolerancias de latencia entre ambas fases operativas:

1. **El planificador de estación terrena (airsim-plan / WebDCS)**: opera de manera previa al despegue del vehículo. En esta etapa, el sistema puede tolerar presupuestos de tiempo del orden de 5 a 10 segundos para interpretar instrucciones en lenguaje natural, consultar información contextual del entorno, aplicar reglas de seguridad operacional y generar un plan de vuelo global validado.
2. **El lazo táctico a bordo (airsim-loop)**: opera en tiempo real estricto una vez que el vehículo está en el aire (capítulo 5). Bajo una frecuencia objetivo de 5 a 10 Hz, su función no es diseñar la ruta global sino mantener la estabilidad cinemática, seguir los objetivos asignados y ejecutar maniobras reactivas inmediatas de evasión de obstáculos a partir de visión monocular ligera.



El desacoplamiento permite que la misión sea validada, persistida y versionada de forma determinista antes de iniciar los motores. Asimismo, garantiza que el lazo táctico a bordo no dependa de una conexión de red permanente con la estación terrena: una vez transmitido el manifiesto de vuelo, el dron opera de forma enteramente autónoma, protegiendo la seguridad del vuelo ante eventuales pérdidas de enlace de radio o telemetría.

4.2 Compilación de misiones desde lenguaje natural

El módulo de planificación en tierra integra un modelo de lenguaje (LLM) que actúa como compilador semántico. Su objetivo es transformar directivas operativas de alto nivel expresadas en lenguaje natural por un operador humano (por ejemplo, "recorrer el perímetro norte a 10 metros de altura evitando zonas de alta densidad vehicular") en un artefacto estructurado y ejecutable por el piloto automático.

Para garantizar la fiabilidad del proceso de compilación sin incurrir en alucinaciones o errores de sintaxis:

- Se utiliza un cliente local compatible con la API de OpenAI (conectado a instancias locales de LM Studio u Ollama), ejecutando modelos orientados a seguimiento de instrucciones (tales como Llama-3-8B, Qwen-7B o Phi-4).
- Se define un **System Prompt** formalizado (`compiler_system.md`) que instruye al modelo sobre las dimensiones del entorno de simulación, las restricciones del espacio aéreo urbano y el formato de salida requerido.
- Se implementa una capa de coerción y extracción de JSON (`json_extract.py`) respaldada por esquemas de validación estricta en **Pydantic** (`manifest.py`). Si la salida generada por el LLM no cumple con los tipos de datos o las restricciones cinemáticas impuestas, el compilador rechaza el plan y solicita una reevaluación antes de comprometer el vuelo.

Cabe una aclaración sobre el alcance actual de la interfaz: si bien el compilador semántico descrito en esta sección está completamente implementado y expuesto como endpoint del backend, la interfaz de WebDCS en su estado presente no ofrece ningún control para invocarlo. La superficie de interacción del operador quedó acotada, por decisión de alcance del proyecto, a la edición manual de waypoints sobre la carta de navegación (§4.4.1) — priorizando el desarrollo del control de vuelo por sobre la interfaz de planificación asistida. La compilación desde lenguaje natural queda documentada aquí como una capacidad ya construida y validada, disponible para reincorporarse a la interfaz sin cambios en el backend.

4.3 El contrato del Manifiesto de Misión (`MissionManifest`)

El producto final del planificador terrestre es un archivo JSON denominado `MissionManifest`. Este documento actúa como un contrato formal e inmutable entre la estación terrena y el lazo táctico de vuelo.

```
{
  "mission_id": "MANHATTAN_URBAN_01",
  "summary": "Patrón de patrullaje urbano sobre cuadrícula con 7 waypoints y retorno seguro.",
  "waypoints": [
    {"x": 0.0, "y": 50.0, "z": -10.0, "label": "wp_start"},
    {"x": 60.0, "y": 50.0, "z": -10.0, "label": "wp_turn_1"},
    {"x": 60.0, "y": 120.0, "z": -10.0, "label": "wp_turn_2"},
    {"x": -40.0, "y": 120.0, "z": -10.0, "label": "wp_target"}
  ],
  "rules_of_engagement": {
    "ignore_objects": ["person", "car"],
    "return_to_launch_battery_threshold": 20.0,
    "max_speed_mps": 5.0,
    "safe_altitude_range_m": [5.0, 30.0]
  }
}
```

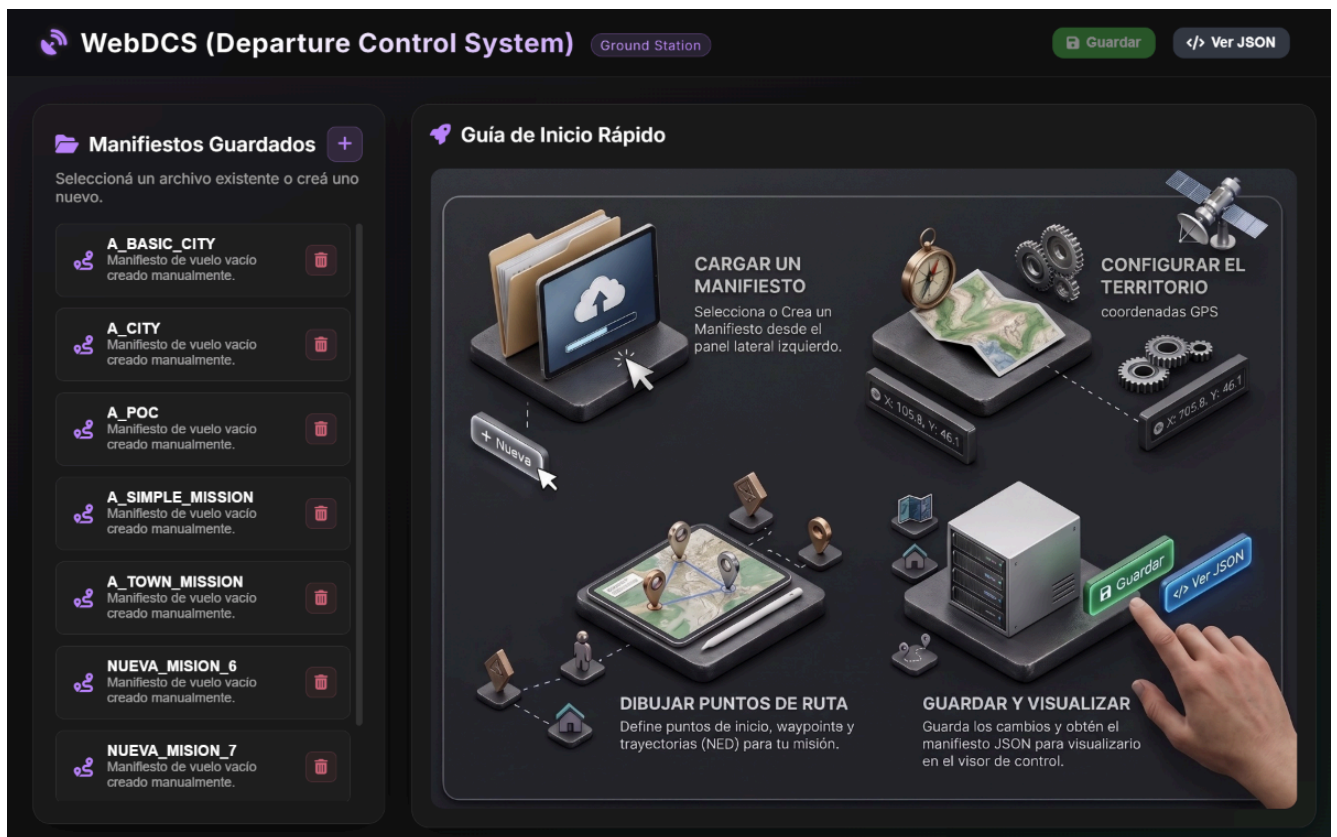
Componentes clave del contrato:

- `mission_id` y metadatos: identificador único y descripción textual de la intención operativa.
- `waypoints`: lista ordenada de puntos de paso tridimensionales expresados en el marco de coordenadas estándar aeronáutico **NED** (*North-East-Down*, donde $z < 0$ representa altitud sobre el punto de despegue).
- `rules_of_engagement`: parámetros operacionales y límites cinemáticos globales, tales como velocidad máxima de crucero (`max_speed_mps`), rangos de altitud permitidos y umbrales de seguridad para retorno automático al punto de lanzamiento (*Return-to-Launch*).

En el marco de la metodología experimental de esta tesis (capítulo 10), el uso de manifiestos estructurados garantiza la **reproducibilidad experimental**: los escenarios de benchmark por Tiers (`minisim_clear`, `townsim_ini`, `citysim_clear`) se definen a través de manifiestos fijos e inmutables, asegurando que las comparaciones de rendimiento entre brazos de control (SLM, FSM y reactivo) se inicien exactamente con las mismas metas cinemáticas y espaciales.

4.4 Estación Terrena WebDCS: planificación y auditoría post-vuelo

Para facilitar la interacción operativa y el análisis experimental posterior, se desarrolló **WebDCS** (*Web-based Drone Control Station*), una estación de control en tierra construida sobre **FastAPI** y tecnologías web estándar (HTML5, Vanilla CSS y JavaScript).



Funcionalidades de WebDCS:

1. **Gestión interactiva de cartas de territorio y waypoints:** La interfaz incorpora un visor basado en Canvas 2D que mapea la geometría del entorno urbano (*CitySim*) a cartas de navegación satelital. Permite a los operadores cargar manifiestos existentes, visualizar las trayectorias proyectadas y editar waypoints interactivamente sobre la carta (agregar, reordenar, eliminar y ajustar la altitud de cada punto).
2. **Auditoría de vuelo post-misión, sincronizada por video:** Cada corrida del lazo táctico registra su propia carpeta autocontenida (identificador de misión y marca de tiempo ISO de inicio) con la traza completa en formato CSV/JSONL, los fotogramas enviados al modelo de lenguaje en cada consulta deliberativa y un video anotado del vuelo completo (un fotograma por ciclo, con overlay de la acción tomada, el nodo del grafo de decisión activo y el estado de los sectores de percepción). Un visor HTML autocontenido, generado automáticamente al cierre de cada corrida, sincroniza en ambos sentidos la reproducción del video con la fila correspondiente de la traza: reproducir el video mueve un cursor sobre la tabla de ciclos, y seleccionar una fila salta el video al instante correspondiente — permitiendo reconstruir exactamente qué fotograma vio el modelo y qué decisión tomó en cualquier punto del vuelo, sin depender de que el operador haya observado la sesión en vivo.

(Una etapa temprana del desarrollo exploró un mecanismo alternativo de transmisión en vivo de video y telemetría hacia la interfaz web, basado en WebSockets y *Server-Sent Events*. El canal quedó sin ningún consumidor tras un rediseño posterior del frontend y fue retirado por completo — tanto el backend como la interfaz — en favor del mecanismo de auditoría post-vuelo aquí descrito, que resultó suficiente para las necesidades de análisis experimental de este trabajo y más simple de mantener.)

1. **Registro íntegro de cada consulta al modelo de lenguaje:** Independientemente del mecanismo de auditoría visual del punto anterior, cada entrada de la traza CSV/JSONL conserva el prompt exacto enviado al modelo, la respuesta cruda emitida, el tiempo de inferencia en milisegundos, la macro-acción resultante y su justificación semántica, además de indicadores de activación de los mecanismos de respaldo deterministas (timeout del watchdog, salida no parseable). Esta traza es la fuente primaria de evidencia para el análisis de modos de falla del capítulo 9 y la metodología experimental del capítulo 10.
2. **Secuencia de fin de misión y aterrizaje autónomo:** Al alcanzar el último waypoint del manifiesto o al solicitarse la detención de emergencia desde la interfaz web, el sistema coordina una secuencia controlada de finalización: ejecución de descenso autónomo (*LandAsync*), corte de motores, liberación segura de los recursos de API del simulador y retorno ordenado a la pantalla de espera de la estación terrena.

5. Arquitectura del lazo táctico

5.0 Escaneo espacial pre-vuelo (`spatial_scan`)

Antes de que el grafo de decisión empiece a ejecutarse, `main.py` llama de forma bloqueante a la función `spatial_scan()` (`src/agents/spatial_scan.py`), implementada como parte de PLAN-MEJORAS-3 (H1). Esta función no es un nodo del `StateGraph`: se ejecuta una sola vez, entre la conexión con AirSim y la construcción del estado inicial del grafo.

El escaneo realiza un barrido de yaw en el lugar (sin traslación) hacia un conjunto de rumbos equiespaciados alrededor del rumbo inicial, captura un fotograma en cada rumbo tras un breve período de asentamiento y envía todas las imágenes en una única consulta al VLM. El modelo devuelve un contexto cualitativo del entorno visible desde el punto de despegue (obstáculos prominentes, corredores potenciales, densidad general de la escena). Este contexto es **advisory, no safety-critical**: el `ObstacleField` por ciclo sigue siendo la única autoridad de seguridad una vez que el dron está en movimiento.

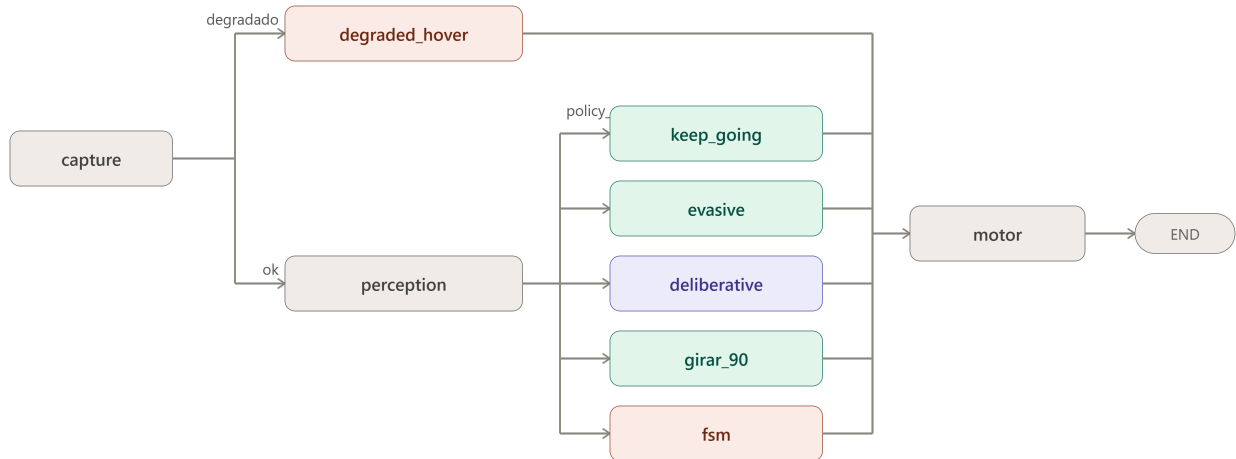
Si el VLM no responde dentro del watchdog del escaneo, o devuelve una respuesta no parseable, la misión arranca igualmente sin contexto inicial — un VLM caído nunca impide el despegue. Al igual que el resto del sistema de percepción (cap. 6), el escaneo pre-vuelo opera exclusivamente sobre fotogramas RGB monoculares y nunca consulta el canal de profundidad del simulador.

La latencia de `spatial_scan` es un costo de inicialización de única vez, no un costo por ciclo; se mide y registra separadamente del presupuesto de latencia táctica (§10.2).

5.1 Visión general del lazo

El lazo descrito en este capítulo se ejecuta sobre el entorno de simulación construido y validado en el capítulo 3 (Unreal Engine 5.5 con Cosys-AirSim), ejecutando los manifiestos compilados previamente por la estación terrena (capítulo 4). El sistema implementado es un grafo de decisión por ciclo (`StateGraph` de LangGraph) que se ejecuta a una frecuencia objetivo de `LOOP_HZ` (5–10 Hz según la resolución de captura elegida). Cada ciclo recorre el mismo grafo desde `capture` hasta `motor` y llega a un estado terminal (`_end__`); la naturaleza cíclica de la navegación la aporta el bucle externo de `main.py`, no el grafo en sí, que por diseño no tiene aristas de retorno. Conviene describir esta arquitectura como un **grafo de decisión por tick** y no como un "grafo cíclico de navegación": es una distinción defendible y no una mera cuestión de vocabulario, porque el grafo se recompila y evalúa desde cero en cada ciclo.

La estructura actual del grafo, exportada directamente desde el código compilado (`scripts/export_graph_mmd.py`, sin conexión a AirSim ni efectos secundarios), es la siguiente:



Cada ciclo captura un fotograma y telemetría (`capture`); si la fuente de datos no es AirSim (modo degradado, ver más abajo), el ciclo va directo a `degraded_hover` sin pasar por percepción. En caso contrario, `perception` produce el `ObstacleField` del ciclo (cap. 6) y un enrutador de política (`policy_router`) decide, sobre esa evidencia, a cuál de cinco nodos de acción dirigir el ciclo: `keep_going` (crucero sin novedad), `evasive` (evasión reactiva de corto plazo), `gitar_90` (bypass determinista ante bloqueo severo), `fsm` (política de máquina de estados, cuando ese es el brazo activo) o `deliberative` (consulta al modelo de lenguaje). Todos los nodos de acción convergen en `motor`, que traduce la macro-acción elegida a un comando cinemático concreto y lo emite a AirSim.

5.2 Deliberación por excepción

El principio de diseño central de la arquitectura es que la consulta al modelo de lenguaje es la **excepción**, no la regla: la mayoría de los ciclos deben resolverse por rutas deterministas y baratas (`keep_going`, `evasive`, `gitar_90`), y solo una fracción acotada de los ciclos debería requerir la latencia y el costo de una inferencia de lenguaje. Ese principio se sostiene con varias salvaguardas explícitas:

- **Avance cauteloso y frenado condicional por seguridad.** Durante la espera asíncrona de la resolución del modelo, el sistema comanda un avance a velocidad mínima de arrastre (`DELIB_WAIT_CREEP_SPEED_MPS = 0.5 m/s`) en lugar de frenar a cero incondicionalmente. Esta distinción cinemática es fundamental: detener por completo la traslación anula la señal del flujo óptico y destruye la confianza del estimador de TTC, desencadenando espirales deliberativas artificiales. Sin embargo, si el sector frontal registra un bloqueo estructural inminente (`close_structural = True`), el sistema comanda detención total inmediata (`FRENAR`), anteponiendo la seguridad física a la preservación del flujo.
- **Deliberación asíncrona con watchdog.** La consulta al modelo corre en un hilo aparte (`DeliberationService`, con una cola de tamaño 1) para no bloquear el lazo de percepción mientras se espera la respuesta. Si el pedido excede `SLM_WATCHDOG_MS`, se marca `timeout` y decide la política de respaldo determinista — el ciclo de control nunca queda a merced de la latencia del modelo.
- **Parser tolerante como red de seguridad.** La respuesta del modelo se solicita con `response_format=json_schema` (decodificación restringida) pero se conserva un parser tolerante a texto conversacional, JSON truncado o markdown como camino de respaldo — nunca se descarta un ciclo completo solo porque la salida no fue perfectamente estructurada (cap. 8).
- **Fallback determinista.** Ante timeout, respuesta no parseable o acción fuera de la lista blanca, el sistema recurre a una política de respaldo determinista, nunca a un valor por defecto arbitrario.
- **Persistencia de maniobra y enclavamiento anti-flip-flop.** Una vez iniciada una maniobra de evasión o escape, el sistema evita revertirla ciclo a ciclo por pequeñas variaciones de la evidencia; y un mecanismo de escape que agota sus reintentos permitidos queda enclavado y cambia de estrategia en lugar de repetir indefinidamente la misma acción fallida.

5.3 Salvaguardas y contratos congelados

Tres estructuras de datos funcionan como contratos estables de la arquitectura, en el sentido de que su superficie de API no cambia aunque cambie su implementación interna:

- `deliberations[]` es el registro de evidencia primaria de cada decisión del modelo de lenguaje (prompt enviado, respuesta cruda, modelo, latencia, si hubo fallback). Es un contrato de solo agregado: se le suman campos (por ejemplo `timeout`, `adherent`) pero nunca se le quitan, porque es la evidencia auditable de la que depende buena parte del capítulo de resultados.
- `ObstacleField` (cap. 6) es el único objeto que consumen el enrutador de política, el nodo de evasión, el nodo deliberativo, la FSM y el registro de vuelo. Ningún consumidor accede a campos crudos de flujo óptico: toda lectura pasa por su API pública (`is_blocked`, `blocked_fraction`, `sector_ttc`, `summary_text`, `to_dict`).
- `WaypointTracker`, con corrección de rumbo por *cross-track error*, zona muerta angular, saturación de tasa de guiñada e histéresis con suavizado exponencial (EMA) sobre los umbrales de giro brusco, aproximación final y zona muerta de guiñada, agregados para reducir cabeceos visibles en el vuelo. Incluye además comportamiento de movimiento vertical puro (`near_vertical`): cuando el siguiente waypoint está directamente arriba o abajo del dron (`dist_xy < 1 m` AND `|dz| > 0.3 m`), se fuerza `vx = 0` para un ascenso o descenso estrictamente vertical. Este comportamiento es el que hace posible el patrón *climb-first* de Tier 2 (§3.2): sin él, el tracker intenta moverse horizontalmente mientras sube, generando una trayectoria diagonal que puede atravesar edificios.

5.4 Modo degradado

Cuando la fuente de datos no es AirSim (`source != "airsim"`), el sistema no sustituye el fotograma o la telemetría faltante por datos sintéticos plausibles: el ciclo se dirige directo a `degraded_hover`, se comanda hover explícito y se omiten percepción y deliberación por completo. La razón de este diseño es la misma que motiva el capítulo 9: un dato sintético "razonable" en lugar de un dato ausente es indistinguible, para el resto del sistema, de un dato real, y esconde exactamente el tipo de fallo silencioso que este trabajo documenta en otros componentes.

5.5 Escalación ante atasco persistente: barrido panorámico y consulta puntual al modelo de visión

Cuando el mecanismo de evasión de corto plazo (`evasive`) no logra restablecer el progreso hacia el waypoint objetivo durante un número sostenido de ciclos — un atasco duro, en el sentido de que ninguna dirección inmediata frente al dron ofrece un corredor despejado según el `ObstacleField` (cap. 6) — el sistema no recurre de inmediato al escape ciego (alternancia determinista entre ganar y perder altitud). En su lugar, ejecuta primero un barrido panorámico: una secuencia de giros puros en el lugar (sin traslación) hacia un conjunto fijo de rumbos equiespaciados alrededor del rumbo actual, capturando un fotograma en cada uno tras un breve período de asentamiento.

Con el panorama completo capturado, se realiza una única consulta al modelo de visión, presentando las imágenes de todos los rumbos explorados — etiquetadas explícitamente como direcciones distintas desde el mismo punto, no como fotogramas consecutivos en el tiempo — junto con el estado del atasco (ciclos sin progreso, escapes verticales ya intentados, distancia y rumbo al objetivo). El modelo elige una única macro-acción de la misma lista blanca que usa el nodo deliberativo ordinario (cap. 8): el barrido no introduce vocabulario de decisión nuevo, solo una vista panorámica de la evidencia. Si la respuesta no llega dentro del plazo del watchdog o no resulta en una acción viable, el sistema cae al escape ciego determinista como red de seguridad final, garantizando que el mecanismo de escalación nunca deje al vehículo sin una decisión.

Todo el barrido se ejecuta dentro del mismo ciclo del grafo de decisión, nunca en un bucle separado: reutiliza el fotograma que el nodo `capture` ya produjo ese ciclo y reemite su propio comando de velocidad a través de `motor` como cualquier otro nodo de política, sin desactivar la vigilancia del enrutador de política mientras dura. Al igual que el resto de la percepción del sistema (cap. 6), el barrido nunca consulta el canal de profundidad del simulador: opera exclusivamente sobre los mismos fotogramas RGB monoculares que alimentan el resto del lazo.

Este mecanismo de escalación es controlable como variable experimental (`DEADLOCK_STRATEGY=blind|deep_vlm`, adoptando `deep_vlm` como valor por defecto en producción): el sistema puede configurarse para omitirlo y recurrir directamente al escape ciego (`blind`), lo que permite comparar de forma factorial ambas estrategias de resolución de atasco bajo idénticas condiciones de vuelo (cap. 10). En corridas de validación sobre el escenario urbano de referencia, la variante con consulta al modelo de visión (`deep_vlm`) resolvió la totalidad de los eventos de atasco duro observados (12/12) sin recurrir al escape ciego de respaldo.

5.6 Registro de vuelo y auditoría post-vuelo

Cada ejecución del lazo táctico — interactiva o lanzada desde la estación terrena (cap. 4) — genera, salvo que se desactive explícitamente, una carpeta autocontenida identificada por el nombre de la misión y una marca de tiempo ISO de inicio. Dentro de ella conviven: una traza en formato JSONL con el estado completo de cada ciclo, la misma traza en CSV para inspección tabular directa, los fotogramas enviados al modelo de lenguaje en cada consulta deliberativa (nombrados por su marca de tiempo real de captura), un resumen agregado por waypoint (ciclos consumidos, proporción de ciclos deliberativos y eventos de escalación por atasco de cada tramo de la misión) y, opcionalmente, un video del vuelo completo con overlay de la acción tomada en cada ciclo.

La traza CSV/JSONL es el contrato de evidencia primaria del sistema: cada fila conserva el prompt exacto y la respuesta cruda de toda consulta al modelo de lenguaje, la telemetría completa del vehículo, el estado del campo de obstáculos por sector y la ruta del grafo de decisión que resolvió ese ciclo — es la fuente de la que se derivan las métricas del capítulo 10 y el análisis de modos de falla del capítulo 9.

Para la inspección cualitativa del comportamiento del sistema en un vuelo particular, cada corrida incluye además un visor HTML autocontenido que sincroniza en ambos sentidos la reproducción del video con la fila correspondiente de la traza: avanzar el video actualiza automáticamente el panel de detalle del ciclo correspondiente (incluyendo el prompt, la respuesta y los fotogramas enviados al modelo en ese instante), y seleccionar una fila de la tabla salta el video al momento correspondiente. Esta herramienta permite reconstruir, para cualquier instante del vuelo, exactamente qué percibió el sistema y por qué tomó la decisión que tomó.

6. Percepción monocular sin redes neuronales

6.1 Fundamentos de diseño: percepción geométrica y física de aproximación

La arquitectura de percepción a bordo implementada en este trabajo prescinde deliberadamente de redes neuronales profundas de detección (como detectores de cajas delimitadoras o segmentadores semánticos) y fundamenta la estimación de obstáculos en visión por computadora clásica: **flujo óptico denso derotado y divergencia del campo traslacional**. Esta decisión se sustenta en tres principios de ingeniería robótica:

- Determinismo y presupuesto de cómputo en tiempo real:** En una plataforma de navegación autónoma donde los recursos de cómputo (GPU/CPU) deben compartirse con la inferencia de un modelo de lenguaje local (SLM), los algoritmos de visión clásica optimizados (`cv2.DISOpticalFlow`) garantizan una latencia determinista y acotada por ciclo (5–10 Hz), eliminando los cuellos de botella de inferencia asociados a redes convolucionales o transformadores visuales densos.
- Generalización universal y robustez fuera de distribución (OOD):** Los detectores de objetos supervisados están limitados a las clases semánticas presentes en sus conjuntos de entrenamiento (automóviles, personas, señales). En un entorno urbano tridimensional complejo, los obstáculos potenciales incluyen cables, salientes arquitectónicas, andamios o geometrías irregulares sin etiqueta previa. El flujo óptico modela directamente el fenómeno físico de aproximación espacial mediante la expansión de textura en el plano focal, reaccionando ante cualquier objeto que represente un riesgo de colisión sin importar su clase semántica ni su apariencia.
- Modelado físico de Tiempo-a-Colisión (TTC):** Una caja delimitadora 2D no provee información cinemática de cierre ni distancia métrica directa. En contraste, la divergencia del campo de flujo traslacional permite derivar matemáticamente el Tiempo-a-Colisión ($TTC \approx Z / v_z$) de forma continua a lo largo del plano de la imagen, entregando una señal temporal interpretable y calibrable para las decisiones de maniobra reactiva.

6.2 Flujo óptico, derotación y divergencia como señal de ocupación

La percepción del sistema produce en cada ciclo un objeto de estado unificado denominado `ObstacleField` (`src/perception/obstacle_field.py`), generado por el estimador de flujo y TTC (`FlowTTCEstimator`, `src/perception/flow_ttc.py`). Los componentes del grafo de control consumen este estado exclusivamente a través de su API pública (`is_blocked`, `blocked_fraction`, `sector_ttc`, `summary_text`, `to_dict`), aislando la lógica de control de los cálculos numéricos de bajo nivel.

El campo espacial se organiza en una grilla de 3×3 celdas (tres sectores horizontales: *izquierda*, *centro*, *derecha*; por tres bandas verticales: *superior*, *medio*, *inferior*). Para cada celda se computan cuatro magnitudes principales: - **Ocupación** (`occupancy`, $\in [0, 1]$): Nivel de actividad de divergencia positiva en la celda. - **Tiempo-a-Colisión** (`ttc_s` en segundos): Estimación de tiempo para el impacto (∞ si no hay aproximación). - **Divergencia traslacional** (`divergence` en $1/s$): Tasa de expansión del flujo traslacional. - **Confianza** (`confidence`, $\in [0, 1]$): Proporción de píxeles válidos con gradiente suficiente en la celda.

El procesamiento por ciclo ejecuta las siguientes etapas:

- Flujo óptico denso:** Cálculo del vector de desplazamiento entre fotogramas consecutivos escalados a una resolución normalizada (`FLOW_DOWNSCALE_WIDTH`), empleando el algoritmo variacional DIS (*Dense Inverse Search*).
- Derotación analítica por telemetría de actitud:** La rotación propia del vehículo (cambios de *roll*, *pitch* y *yaw* registrados por la IMU entre fotogramas) genera un flujo óptico angular parásito que enmascara la aproximación real. El estimador proyecta analíticamente la velocidad angular sobre el plano focal y resta este componente del flujo total medido: $\mathbf{v}_{trans} = \mathbf{v}(\Delta\phi, \Delta\theta, \Delta\psi)$. Esta corrección garantiza que giros puros de guiñada o cabeceos de estabilización no generen falsas alarmas de colisión frontal. - $\mathbf{v}_{rot}$
- Estimación del Foco de Expansión (FOE):** Localización del punto de fuga del vector de traslación mediante mínimos cuadrados ponderados con eliminación recursiva de valores atípicos (*outliers*). Si el dron se encuentra en estacionario (*hover*) o movimiento lateral puro, el FOE se marca como indefinido y el TTC se asigna a ∞ .
- Cálculo de TTC por celda:** Para cada píxel válido p , se calcula $TTC(p) = \frac{|p - \mathbf{FOE}| \cdot \Delta t}{\mathbf{v}_{trans}(p)}$. La agregación a nivel de celda toma el percentil 20 de la distribución interna, proporcionando una estimación conservadora y resistente al ruido espurio.

6.3 El canal de ocupación y calibración de escala

La divergencia del campo traslacional ($\nabla \cdot \mathbf{v}_{trans}$) se evalúa en `src/perception/flow_ttc.py` mediante operadores diferenciales sobre las componentes horizontal y vertical del flujo. En la implementación de referencia, la ocupación se deriva escalando la divergencia positiva (`occupancy = clip(divergencia × 0.5, 0, 1)`).

El predicado de bloqueo por celda `Cell.is_blocked()` integra de forma complementaria ambos canales de percepción mediante una compuerta lógica disyuntiva: $\text{is_blocked} = (\text{occupancy} \geq \tau_{occ}) \vee (\text{ttc_s} \leq \tau_{ttc})$ donde $\tau_{ttc} = 3.2s$ corresponde al umbral óptimo calibrado frente a profundidad de referencia (capítulo 7) y τ_{occ} es el umbral de ocupación (`OBSTACLE_OCCUPANCY_BLOCKED`). La calibración formal de este operador y su escala frente a datos de profundidad constituye un punto de análisis metodológico relevante (capítulo 7 y 9).

6.4 Métrica de bloqueo global: `blocked_fraction()`

Para la toma de decisiones a nivel macro en el grafo de navegación, `ObstacleField.blocked_fraction()` computa la proporción de celdas bloqueadas sobre el total de la grilla 3×3.

Cuando esta fracción supera el umbral crítico de bloqueo generalizado, el sistema determina que la trayectoria frontal se encuentra completamente comprometida en todos sus sectores, transfiriendo el control al nodo determinista `girar_90` para efectuar una maniobra de escape ortogonal inmediata sin incurrir en latencias de deliberación innecesarias.

7. Estimación de TTC y su validación

7.1 Protocolo de validación

El canal de profundidad de AirSim (`DepthPlanar`, en metros) permite construir una referencia (*ground truth*) del tiempo-a-colisión sin necesidad de instrumentación adicional, y sin costo de vuelo extra. `experiments/collect_ttc_dataset.py` recolecta, por ciclo y por celda, el TTC estimado, el TTC de referencia, la divergencia, la ocupación, la confianza, y variables de contexto (velocidad, tasa de guiñada, algoritmo de flujo usado). El TTC de referencia por celda se calcula como

$$\text{TTC}_{\text{celda}} = \text{percentil20}(Z_{\text{celda}}) / \max(v_{\text{closing}}, \epsilon)$$

donde Z_{celda} es la profundidad de la celda (percentil 20, para ser robusto a ruido puntual) y v_{closing} es la componente de la velocidad del cuerpo del dron (de telemetría) proyectada sobre el eje óptico de la cámara.

El conjunto de datos recolectado (3735 registros, `runs/ttc/*.jsonl`) cubre tres escenarios scripteados: aproximación frontal a un edificio, vuelo recto por un cañón urbano, y giros de guiñada sin componente de aproximación — diseñado específicamente para ejercitar la derotación de guiñada descrita en el capítulo 6, desacoplando el giro propio de la señal de aproximación frontal.

7.2 Resultado y su interpretación correcta

El análisis (`experiments/analyze_ttc.py`) reporta dos resultados que, leídos por separado, parecen contradictorios: la correlación puntual entre el TTC estimado y el TTC de referencia es floja ($r \approx -0.034$, error relativo mediano del 66 %), pero el área bajo la curva ROC (AUC) para el evento binario "colisión dentro de τ segundos" (con $\tau \in \{1, 2, 3\}$) es de 0.96–0.97, y los umbrales elegidos por el índice de Youden son $\tau=2 \text{ s} \rightarrow \text{TTC}=3.18 \text{ s}$ y $\tau=3 \text{ s} \rightarrow \text{TTC}=4.58 \text{ s}$.

La lectura correcta de estos dos números en conjunto es que el estimador **no es un cronómetro**: no hay que interpretar "TTC = 3.2 s" como una medición física precisa de cuánto tiempo falta para una colisión. Lo que sí es, con una AUC de 0.96–0.97, es un **ordenador de riesgo monótono confiable**: un valor de TTC más bajo corresponde, de forma consistente, a mayor riesgo de colisión próxima, aunque el valor puntual en segundos no deba tomarse como magnitud física exacta. Presentarlo como "un puntaje de riesgo calibrado por curva ROC, cuyo umbral tiene unidades de segundos por construcción pero no interpretación métrica directa" es la formulación defendible; presentarlo como "TTC = 3.2 s" sin ese matiz sería sobre-especificar la precisión del estimador. Con esos umbrales, `TTC_EVASION_THRESHOLD` se fijó en 3.2 s (antes, un valor provisorio de 3.0) y `TTC_SAFE_THRESHOLD` en 4.6 s (antes, 6.0), ambos derivados de los valores de Youden y no elegidos a mano.

Una limitación queda explícitamente documentada: la validación proviene de una única sesión de vuelo en simulador, y el escenario de giros de guiñada quedó, en la práctica, casi enteramente concentrado en el rango $|\text{yaw_rate}| \in [0, 0.05]$ rad/s. Eso valida bien la detección frontal, pero **no** valida todavía la derotación en giros agresivos con datos reales — es precisamente el caso de uso para el que se diseñó la derotación (cap. 6), y su validación con un rango más amplio de tasas de guiñada queda como trabajo pendiente.

7.3 Calibración del canal de ocupación (pendiente)

A diferencia del TTC, el canal de ocupación de `ObstacleField` no pasó todavía por este mismo protocolo de validación contra profundidad. Como se documenta en el capítulo 6 (sección 6.3), el cálculo actual de la divergencia tiene un error de escala conocido (kernel de Sobel sin normalizar), lo que hace que calibrar el umbral de ocupación contra el canal de profundidad —con el mismo enfoque de curva ROC ya aplicado al TTC— sea un paso pendiente y no meramente un refinamiento opcional: el umbral actual (`OBSTACLE_OCCUPANCY_BLOCKED`) sigue siendo un valor provisorio, tal como lo señala el propio código.

TTC real	Divergencia reportada	Occupancy media	% de celdas $\geq$ umbral
pendiente de recalibración	pendiente	pendiente	pendiente

7.4 Validación de la derotación con giros agresivos (pendiente)

Ampliar el escenario de giros de guiñada del conjunto de datos de validación (sección 7.1) para cubrir tasas de guiñada más agresivas (del orden de ± 0.3 a ± 0.5 rad/s, sin componente de traslación), y re-correr la estratificación del error de TTC por `|\text{yaw\_rate}|`, permitiría confirmar si el error relativo en los bins de guiñada alta es comparable al de guiñada baja. Si no lo es, indicaría un error de signo o una desincronización entre la telemetría de actitud y el fotograma que debería corregirse antes de dar por cerrada la validación del canal de TTC. Esta ampliación queda como trabajo pendiente.

8. Ingeniería de decisiones del SLM

8.1 Selección del modelo

La restricción de hardware determina buena parte de las decisiones de este capítulo: compartir GPU con una simulación de Unreal Engine 5.5 deja un presupuesto de VRAM acotado para la inferencia a bordo, lo que orienta la elección hacia modelos multimodales compactos (*Small Vision-Language Models*, VLM) del orden de 3 a 7 mil millones de parámetros. En la implementación definitiva, el sistema adopta **Qwen2.5-VL-3B-Instruct** (ejecutado localmente bajo LM Studio o vLLM), el cual combina capacidad de razonamiento simbólico con comprensión visual directa de la cámara a bordo.

A diferencia de un SLM textual puro que únicamente lee resúmenes serializados de la percepción, el nodo deliberativo multimodal procesa:

1. **Historial temporal de fotogramas** (`VLM_FRAME_HISTORY_SIZE = 2`): Envío conjunto de los fotogramas en tiempo real correspondientes al ciclo actual (`$t$`) y al ciclo previo (`$t-1$`), permitiendo al modelo estimar la dirección visual de aproximación o desvío.
2. **Dinámica de vuelo y actitud**: Inyección de la velocidad horizontal estimada y los ángulos de orientación física (`pitch` y `roll`) reales, obtenidos tras la corrección del cuaternión en `cosysairsim`, alertando explícitamente si el cuadricóptero se encuentra en traslación activa o prácticamente detenido.
3. **Motivo explícito de consulta**: Distinción estructurada en el prompt entre consultas forzadas por un obstáculo frontal severo con TTC bajo frente a consultas disparadas por pérdida transitoria de confianza del flujo óptico (evitando que el modelo alucine bloqueos inexistentes en maniobras de rotación sobre el eje).

El detalle de modelos preliminares explorados (Phi-4-mini-instruct, Qwen-Coder-7B, entre otros), su huella de VRAM estimada y el procedimiento de configuración quedan documentados en `informe/anexos/A1-EXPLORACION-SLM-GGUF.md` y `informe/anexos/A2-SLM-CONCEPTO-Y-VENTAJAS.md`.

Es relevante señalar, para el posicionamiento de este trabajo frente al estado del arte (cap. 2), que la arquitectura de lazo cerrado estado→SLM→comando→ejecución no es, hacia 2025-2026, especialmente novedosa: es uno de los patrones ya estandarizados en investigación y prototipos aplicados de UAVs controlados por modelos de lenguaje (ver `informe/anexos/A1-EXPLORACION-SLM-GGUF.md`). El aporte de este trabajo no está en la novedad de esa arquitectura general, sino en la ingeniería de su interfaz con la percepción (cap. 6) y en la documentación medida de sus modos de falla (cap. 9).

8.2 Decodificación restringida vs. parser tolerante

La respuesta del modelo se solicita con `response_format={"type": "json_schema", ...}` —soportado tanto por LM Studio como por Ollama— en lugar de depender únicamente de un parser por expresiones regulares sobre texto libre. La decodificación restringida reduce la superficie de respuestas inválidas o verborágicas, pero no sustituye por completo al parser: `_parse_decision()` se conserva como red de seguridad, capaz de tolerar markdown, texto conversacional o JSON truncado, precisamente porque la garantía de `json_schema` depende de que el backend local la soporte en la versión efectivamente desplegada, algo que no puede darse por sentado sin verificarlo en cada entorno.

La métrica que resume esta capa es `adherence_rate`: la fracción de respuestas parseables al primer intento, medida con y sin decodificación restringida. Es, en sí misma, una fila de la tabla de resultados (cap. 11) y no solo una decisión de ingeniería: cuantifica cuánto aporta, en la práctica, la gramática restringida por sobre el parser tolerante como única defensa.

8.3 Espacio de acción discreto

El modelo de lenguaje —al igual que la FSM y el brazo puramente reactivo (cap. 10)— no produce comandos cinemáticos directamente: elige entre un conjunto acotado y auditable de macro-acciones (`keep_going`, `evasive`, `girar_90`, y las que resuelve el nodo deliberativo o la FSM según el brazo activo, incluida una ruta `fsm` cuando ese es el brazo seleccionado, y un estado `degraded` para modo degradado). Restringir la salida a una lista blanca de acciones válidas — en lugar de permitir que el modelo produzca parámetros cinemáticos libres — es la decisión de diseño que hace posible, a la vez, la decodificación restringida de la sección 8.2 y la comparación limpia entre brazos del capítulo 10: los tres brazos comparten el mismo espacio de acciones y el mismo traductor a comandos (sección 8.4), de modo que lo único que difiere entre ellos es **quién** elige la etiqueta, no **qué puede** elegir.

8.4 `action_to_command`: la frontera entre lenguaje y cinemática

Cada macro-acción se traduce a un comando cinemático concreto (velocidades lineales en los tres ejes y tasa de guiñada) a través de una única función centralizada (`action_to_command`, `src/agents/action_map.py`), compartida de forma estricta por los nodos deliberativo, de evasión y de FSM.

Esta frontera desacopla la semántica de alto nivel del control de bajo nivel y garantiza un invariante crítico de diseño: **ninguna macro-acción puede poseer definiciones cinemáticas dispares o no coordinadas en el sistema**. Al centralizar la traducción, se previene la introducción de derivas asimétricas no deseadas en maniobras de evasión o ascenso (capítulo 9), asegurando que tanto las decisiones generadas por el modelo de lenguaje como las derivadas por la máquina de estados o el control reactivo se ejecuten bajo idénticas restricciones y perfiles de velocidad.

8.5 Nota: LoRA como alternativa no adoptada

El pipeline final usa el SLM tal como se distribuye en formato GGUF cuantizado, sin ajuste fino. LoRA (*Low-Rank Adaptation*) se evaluó en la etapa de planificación como una posible vía de especialización del modelo —documentada en [informe/anexos/A4-OPTIMIZACION-LoRA.md](#)— pero no hay evidencia en el código actual de que se haya implementado en el pipeline final. Se deja constancia explícita de que es una alternativa explorada y no adoptada, por decisión de alcance y no por omisión.

9. Modos de falla de lazos de control híbridos con LLM

9.1 El patrón común: integridad de contratos de interfaz

Este capítulo desarrolla el argumento central declarado en la introducción (§1.4): en un sistema robótico híbrido donde un modelo de lenguaje consume descripciones estructuradas del entorno para emitir decisiones de control, el modo de falla más crítico no radica en el razonamiento sintáctico del modelo, sino en la **consistencia e integridad del contrato de datos** que lo alimenta.

Los modelos de lenguaje poseen una notable capacidad para racionalizar cualquier entrada estructurada y emitir respuestas plausibles, bien formadas y ejecutables, aun cuando las premisas subyacentes provengan de campos nulos, sensores degradados o datos descalibrados. Este capítulo documenta una taxonomía de cinco modos de falla estructurales identificados experimentalmente en la arquitectura híbrida, analizando su origen, su impacto en la toma de decisiones y los mecanismos de contención implementados.

9.2 Instancia 1 — Desincronización de esquemas y campos ciegos (Schema Drift)

Una de las fallas más críticas en pipelines multimodales ocurre cuando existe una desincronización de contrato (*contract drift*) entre los productores de percepción y los módulos consumidores (enrutador de política, mecanismo de respaldo determinista y constructor del prompt del SLM).

Cuando un campo de estado de obstáculos permanece en un valor nulo por defecto (por ejemplo, una lista vacía `detected_obstacles = []`), los módulos consumidores interpretan semánticamente ese valor como "espacio aéreo totalmente despejado" en lugar de "sensor no disponible o no calibrado". En consecuencia: 1. El generador de contexto construye un prompt que afirma al modelo de lenguaje que la trayectoria frontal carece de obstáculos. 2. El SLM, actuando con perfecta coherencia lógica respecto de la información provista, comanda velocidad de crucero constante (`keep_going`). 3. El sistema no genera excepciones de ejecución ni errores en los registros, pero el dron navega a ciegas hacia los obstáculos físicos de la escena.

Mecanismo de contención: La arquitectura resuelve este modo de falla unificando toda la percepción en el contrato tipado `ObstacleField` (capítulo 6). Si la estimación de flujo o telemetría no es confiable, el campo reporta explícitamente `confidence = 0` y `ttc_s = inf`, forzando al lazo de control a ingresar en modo de maniobra determinista segura o `degraded_hover` (capítulo 5), impidiendo que la ausencia de señal se disfraze de ausencia de peligro.

9.3 Instancia 2 — Desalineación temporal en secuencias contextuales

El constructor de prompts para modelos de lenguaje multimodales o secuenciales frecuentemente incluye etiquetas temporales relativas (`[Fotograma t-2]`, `[Fotograma t-1]`, `[Fotograma t]`) para facilitar el análisis de velocidad de aproximación. Si el buffer histórico subyacente (`frame_history`) no se gestiona como una estructura de anillo sincrónica con marcas de tiempo verificadas, pueden inyectarse fotogramas estáticos duplicados o no sincronizados bajo rótulos de secuencia temporal activa.

En este escenario, el modelo de lenguaje intenta inferir vectores de movimiento a partir de una secuencia temporal inexistente o artificialmente congelada.

Mecanismo de contención: Implementación de un *ring buffer* estricto con validación de estampas de tiempo ($\Delta t > 0$) y adaptación dinámica del prompt: las etiquetas secuenciales solo se generan si el buffer cuenta con fotogramas temporales efectivamente diferenciados y verificados.

9.4 Instancia 3 — Descalibración de escala en operadores diferenciales y saturación disyuntiva

Como se documenta en el capítulo 6 (§6.3) y el capítulo 7 (§7.3), el cálculo de derivadas espaciales mediante operadores de gradiente discretos (como Sobel de 3×3 sin factor de normalización $1/8$) induce un factor de escala de amplificación sobre la divergencia estimada.

Cuando la regla de decisión integra la ocupación y el Tiempo-a-Colisión mediante una compuerta lógica disyuntiva (OR): $\text{is_blocked} = (\text{occupancy} \geq \tau_{\text{occ}}) \vee (\text{ttc_s} \leq \tau_{\text{ttc}})$ la saturación artificial del canal de ocupación anula la influencia del canal de TTC calibrado, provocando declaraciones sistemáticas de bloqueo ante variaciones menores de textura visual.

Mecanismo de contención: Normalización analítica de los kernels diferenciales y calibración cruzada de los umbrales de activación mediante curvas ROC contra canales de profundidad de referencia (capítulo 7).

9.5 Instancia 4 — Fallas compuestas en orquestadores de grafos (State Clipping y Ciclos Límite)

El análisis de ejecución del grafo de control por ciclo identificó cuatro vulnerabilidades estructurales en la propagación de variables de estado entre nodos, ninguna de las cuales arrojaba excepciones en tiempo de ejecución:

1. **Claves de control descartadas silenciosamente por el esquema del grafo.** LangGraph construye los canales de estado a partir de un `TypedDict` y descarta, sin aviso, cualquier clave que un nodo escriba pero que no esté declarada en ese esquema. Tres claves de control —una que debía reiniciar el contador de progreso estancado, una que acumulaba la memoria corta de resultados de decisiones previas, y una que inyectaba sub-waypoints de esquina— cruzaban la frontera entre nodo y lazo sin estar declaradas, y se perdían en cada invocación del grafo. La consecuencia más grave fue que el contador de ciclos sin progreso nunca se reiniciaba, y crecía de forma monótona.

2. **Un ciclo límite de período 3 en la propia red de seguridad.** El mecanismo pensado para limitar los reintentos de una maniobra de escape frenaba el vuelo pero, en la misma rama de código, también reseteaba a cero el contador de reintentos — es decir, la red de seguridad se reseteaba a sí misma, convirtiendo lo que debía ser un estado terminal en un ciclo que se repetía indefinidamente.
3. **Una métrica de progreso auto-inconsistente por unidades mezcladas.** El umbral de "ciclos sin progreso" se expresaba en ciclos, y el umbral de distancia mínima de progreso se expresaba en metros, definidos de forma independiente entre sí; combinados, exigían una velocidad de acercamiento sostenida mayor a la que el guiado en un giro cerrado podía físicamente entregar. El resultado era que el mecanismo de "atasco" se disparaba de forma prácticamente garantizada a los pocos ciclos de iniciar la misión, sin que hubiera ningún obstáculo real de por medio.
4. **Un mecanismo de escape ciego a la percepción.** El enrutador de decisiones activaba la rama de escape por atasco antes de consultar el campo de obstáculos, así que en varios ciclos consecutivos en los que la percepción reportaba con evidencia válida un sector lateral despejado, el dron siguió ascendiendo de todos modos: la evidencia de que existía una salida disponible no llegaba a considerarse antes de decidir.

La combinación de estos cuatro problemas, más una macro-acción de ascenso con una deriva lateral no justificada en su definición cinemática (corregida junto con el resto de esta cadena; ver §8.4), produjo en una corrida de validación un ascenso acumulado del orden de 356 metros en una sola misión, sin que el sistema lo reportara como una falla — desde la perspectiva de las métricas agregadas de esa corrida, era simplemente una misión que no llegó a destino, no un ciclo de control atrapado en un estado absorbente. La corrección de estos cuatro problemas —declarar explícitamente las claves de estado, hacer que el reintento agotado enclave y cambie de estrategia en lugar de resetearse, medir el progreso por distancia horizontal en vez de tridimensional, y consultar la evidencia de percepción antes de decidir el escape— está incorporada en la arquitectura descrita en el capítulo 5.

9.6 Instancia 5 — Degradaciones silenciosas en el canal de visión y telemetría de actitud

La transición hacia una deliberación multimodal directa (VLM) expuso una quinta categoría de fallas de contrato, vinculadas a la fidelidad sensorial de los datos crudos presentados al modelo:

1. **Inversión cromática persistente (RGB vs. BGR):** Durante meses de experimentación, el cliente de captura de simulación entregó el buffer de imagen de Unreal Engine en formato RGB nativo a un pipeline que asumía la convención BGR de OpenCV. En consecuencia, el modelo de visión deliberó sistemáticamente sobre imágenes con canales rojo y azul invertidos (fachadas terracota observadas en tonos fríos azulados y cielos amarillentos). El VLM jamás emitió una queja sintáctica ni interrumpió su servicio: procesaba los fotogramas y emitía decisiones plausibles a partir de un espectro cromático falso.
2. **Contaminación por primitivas de depuración en el espacio 3D:** Trazas visuales dibujadas en el mundo virtual por scripts de inspección previa (`simPlotLineStrip` en Unreal Engine) no se limpiaban al iniciar el vuelo. La cámara a bordo las capturaba como franjas geométricas reales en la escena, induciendo al VLM a interpretar líneas de depuración como cables u obstáculos físicos delante del dron.
3. **Supresión espuria de ángulos de actitud:** Debido a que el binding `cosysairsim` no implementaba `to_eularian_angles`, los ángulos de *pitch* y *roll* permanecían fijados en 0.0° en cada prompt, privando al modelo de conocer la inclinación física real del vehículo durante maniobras de aceleración.

Mecanismos de contención: Corrección de la transposición de canales en memoria en el punto de captura (`AirSimClient.capture()`), llamada obligatoria de limpieza de primitivas (`simFlushPersistentMarkers()`) al inicializar la conexión, y cálculo matemático directo de ángulos de Euler a partir del cuaternión de orientación.

9.7 Por qué el sistema seguía volando y el modelo seguía respondiendo

Ninguno de los cinco modos de falla descritos en este capítulo se manifestó como un error visible en tiempo de ejecución: en las instancias multimodales y de prompt, el modelo de lenguaje seguía recibiendo entradas bien formadas y devolvía decisiones estructuradas y plausibles; en la orquestación del grafo, el lazo táctico seguía ejecutándose sin excepciones, ciclo tras ciclo. En todos los casos, el sistema "funcionaba" en el sentido más superficial de la palabra — no se caía, no arrojaba errores — mientras tomaba decisiones sistemáticamente equivocadas por razones que solo se hacían visibles leyendo el contrato entre quien produce un dato y quien lo consume, nunca observando el comportamiento agregado del vuelo. Es, en conjunto, un resultado metodológico sobre cómo depurar lazos de control híbridos con modelos de lenguaje, no una lista de anécdotas de desarrollo: la observación del comportamiento —la herramienta de depuración más natural para un sistema que "parece" razonar— es precisamente la que menos sirve para encontrar esta clase de error.

10. Metodología experimental

10.1 Diseño experimental

La evaluación compara tres brazos de decisión sobre el mismo `ObstacleField` (cap. 6), el mismo espacio de macro-acciones y el mismo traductor a comandos cinemáticos (`action_to_command`, cap. 8):

- `s1m` — el modelo de lenguaje multimodal (VLM, típicamente Qwen2.5-VL-3B) como capa táctica y deliberativa (cap. 8). Durante la espera de respuesta del modelo, el sistema aplica un avance cauto a velocidad reducida (`DELIB_WAIT_CREEP_SPEED_MPS = 0.5 m/s`) en lugar de un frenado total, preservando la traslación necesaria para que el estimador de flujo óptico mantenga confianza; solo se comanda detención total incondicional ante un bloqueo frontal inminente y confirmado (`close_structural`). El modelo delibera de forma asíncrona bajo un watchdog de seguridad con respaldo determinista ante timeout o respuestas no conformes, recibiendo un historial temporal de fotogramas ($\$t\$$ y $\$t-1\$$), notas cinemáticas (`pitch`, `roll`, velocidad) y el motivo explícito de la consulta.
- `fsm` — una máquina de estados finitos explícita (`CRUISE → AVOID_LEFT | AVOID_RIGHT | CLIMB | BRAKE → CRUISE`), con transiciones gobernadas por umbrales fijos sobre el mismo `ObstacleField`. Es la línea de base directa contra la que se evalúa la hipótesis de la tesis: alta predictibilidad y costo computacional mínimo, pero sin capacidad de razonamiento contextual ni interpretación semántica del entorno.
- `reactive` — navegación guiada al waypoint sin evasión de obstáculos: cota inferior de rendimiento para desacoplar qué porción del desempeño proviene del lazo táctico de evasión y cuánto del seguimiento cinemático de base.

Adicionalmente, el diseño incorpora como factor experimental cruzado la **estrategia de resolución de atascos** (`DEADLOCK_STRATEGY`, cap. 5), compartida por los brazos `s1m` y `fsm`: - `blind`: maniobra reactiva de escape cinemático tradicional (ganancia de altura o rotaciones de desenganche en bucle abierto). - `deep_vlm`: barrido panorámico espacial multifotograma ejecutado dentro del lazo y consulta deliberativa al VLM para identificar visualmente un corredor despejado antes de forzar el escape ciego como último recurso.

Escenarios de prueba y jerarquía de Tiers

Superando las formulaciones exploratorias preliminares (como los borradores `manhattan_a` y `manhattan_b`, descartados por inconsistencias topográficas y de mapas en el simulador), el protocolo define una jerarquía formal en tres niveles crecientes de dificultad ambiental:

- Tier 0 (Línea de base / Control):** Escenario `minisim_clear` sobre el mapa `crater.png` (MiniSim). Terreno despejado sin obstáculos para cuantificar el guiado puro, la cota inferior de latencia de ciclo y el consumo cinemático ideal.
- Tier 1 (Entorno intermedio / Vegetación y morfología orgánica):** Escenarios sobre el mapa `townsim_calib.png` (TownSim), abarcando la suite de pruebas `T-CALIB-0` a `T-CALIB-5` y la misión de recorrido perimetral `townsim_ini`. Incluye específicamente el escenario `T-CALIB-2` (`townsim_calib_cruce_frontal.json`), diseñado para imponer un **bloqueo frontal masivo genuino** frente a una fachada continua, forzando la activación de las ramas deliberativas y de resolución de atascos.
- Tier 2 (Entorno complejo / Cañones urbanos y corredores angostos):** Escenarios sobre `citysim_calib.png` (CitySim). El escenario base validado para el batch G4 es `citysim_clear.json` (perímetro de manzana con patrón `climb-first` a ~ 70 m AGL); los escenarios con corredores angostos (`citymap_pilot.json` y `citymap_a.json`) corresponden a la fase siguiente del batch.

Como principio metodológico estricto, **se descarta el teletransporte cinemático** (`start_pose` inhabilitado): todas las misiones despegan desde el `PlayerStart` nativo del nivel en Unreal Engine y ejecutan un ascenso vertical controlado previo a la navegación horizontal. A partir de las velocidades medidas en vuelo real (~ 2 a 3 m/s) y la cadencia táctica (5 Hz), el presupuesto temporal por corrida se dimensiona entre **600 y 900 segundos** para permitir la resolución completa de tramos complejos sin truncamiento artificial.

El criterio de arranque para el batch completo exige la validación previa de misiones piloto exitosas (`success = True`, 0 colisiones) en cada nivel. Se ejecutan al menos **cinco semillas por celda factorial** ($\$ \text{Brazo} \times \text{Estrategia de Atasco} \times \text{Tier} \$$) mediante el runner automatizado headless (`experiments/runner.py`).

10.2 Métricas reportadas

El sistema registra y evalúa las siguientes métricas cuantitativas:

- Eficacia y seguridad de vuelo:**
 - Tasa de éxito de misión (`success_rate`, arribo a todos los waypoints dentro del presupuesto).
 - Tasa de colisiones totales por misión y normalizadas por kilómetro recorrido.
 - Distancia mínima a obstáculo, percentil 5 (`min_obstacle_dist_m`), medida mediante el canal de profundidad a cadencia reducida con guardia arquitectónica de no contaminación sobre el lazo de control (cap. 5).
 - SPL (*Success weighted by Path Length*): éxito ponderado por la razón entre la longitud de la trayectoria óptima y la efectivamente recorrida.
 - Tiempo y ciclos totales a destino.
- Dinámica del lazo táctico y latencias:**
 - Latencia total por ciclo ($\$p50\$$ y $\$p95\$$), desagregada por brazo y por nodo activo del grafo de control (`reactive`, `evasive`, `deliberative`, `gírar_90`, `fsm`, `deep_scan`). La latencia del escaneo espacial pre-vuelo (`spatial_scan`, $\$5.0$) es un costo de inicialización de única vez, registrado

por separado del presupuesto por ciclo.

- `deliberation_rate` (fracción de ciclos tácticos que invocan activamente al modelo de lenguaje) e histograma integral de rutas por corrida.
- **Comportamiento del modelo de lenguaje (SLM/VLM):**
 - Número de invocaciones efectivas por misión (contabilizadas por transición de identificador único).
 - Tasa de *fallback* determinista ante respuestas inválidas o fuera de esquema.
 - Tasa de *timeout* del watchdog asíncrono.
 - Tasa de adherencia sintáctica (`adherence_rate`), con y sin decodificación gramatical estructurada (§8.2).
- **Resolución de atascos (Ablation H2/H3):**
 - Tasa de resolución de atascos mediante escaneo profundo (`deep_scan_resolution_rate`).
 - Promedio de ciclos consumidos para resolver el atasco (`deep_scan_avg_cycles_to_resolve`).
 - Tasa de caída a escape ciego de respaldo (`deep_scan_fallback_rate`).
- **Desglose espacial por Waypoint:**
 - Métricas segmentadas por tramo de misión (`<stem>.summary_by_wp.csv`), permitiendo aislar el comportamiento en tramos de ascenso inicial vertical de las fases de cruce y de maniobra horizontal.

10.3 Protocolo estadístico

La comparación entre brazos y configuraciones se realiza mediante pruebas no paramétricas (prueba U de Mann-Whitney para comparaciones pareadas y Kruskal-Wallis para factores múltiples) calculadas sobre las distribuciones de las semillas de cada combinación, reportando además el tamaño de efecto (Cliff's Delta o correlación de rango biserial).

El análisis no se limita a valores agregados globales, sino que evalúa el rendimiento **desagregado por tipo de escenario y morfología de obstáculo**. Esta formulación permite discernir con precisión cuándo la capacidad de interpretación contextual del VLM proporciona ventajas medibles (por ejemplo, al anticipar salidas en pasajes bloqueados o seleccionar corredores abiertos entre vegetación) y en qué situaciones una heurística rígida (FSM) alcanza un rendimiento equivalente a una fracción mínima del costo computacional.

10.4 Reproducibilidad, auditoría y cuarentena de datos

El subsistema de registro (`src/logging/flight_logger.py`, `flight_video.py`, `flight_viewer.py`) genera por cada misión un directorio autónomo de auditoría (`airsim-runs/<mission_id>-<timestamp>/`) que comprende:

1. **Telemetría granular:** Archivo `JSONL` estructurado ciclo a ciclo y archivo `CSV` correspondiente para análisis tabular directo.
2. **Resúmenes agregados:** `summary.json` con los indicadores globales de la corrida y `summary_by_wp.csv` con el desglose por waypoint.
3. **Auditoría visual forense del VLM:** Almacenamiento individual en formato `PNG` (`photo-<timestamp_ISO>.png`) de cada fotograma exacto presentado al modelo, preservando el timestamp de captura real y evitando discrepancias de inferencia.
4. **Grabación de video continua sincronizada:** Archivo `.webm` (códec VP8 sin dependencias de licencias propietarias) grabado a la tasa nominal `L00P_HZ`, con superposición en pantalla de la telemetría cinemática, lecturas sectoriales de TTC y el nodo activo del grafo.
5. **Visor interactivo offline (`viewer.html`):** Aplicación web embebida en la carpeta de la corrida que vincula un deslizador temporal con el video y la tabla de telemetría, permitiendo inspeccionar sincronizadamente el prompt textual, los fotogramas y la respuesta del modelo en cada decisión.

Trazabilidad y cuarentena experimental: Cada corrida incorpora automáticamente el hash de commit de Git (`code_version`) en `summary.json`. Para garantizar la validez científica de los resultados, se establece una **política estricta de cuarentena**: ningún vuelo anterior al 2026-09-03 se incluye en el análisis estadístico formal del capítulo 11, dado que los registros previos estuvieron expuestos a artefactos instrumentales ya corregidos (inversión de canales de color R/B en la captura, persistencia de líneas de depuración en el visor 3D y supresión espuria de los ángulos de *pitch* y *roll*).

El conjunto de validación de TTC (cap. 7), junto con los archivos de telemetría y configuración del batch definitivo, serán archivados y publicados con un identificador digital persistente (DOI vía Zenodo), garantizando la completa auditabilidad y reproducibilidad de la tesis.

11. Resultados comparativos SLM vs. FSM

Estado (2026-09-07): corridas piloto de Tier 0, Tier 1 base y Tier 2 base completadas (1 semilla, ilustrativo). El batch estadístico G4 (≥5 semillas, análisis Mann-Whitney) está pendiente de `townsim_ini` y `citymap_a`. Las tablas de esta sección presentan datos reales de las corridas piloto en §11.1 y marcadores de posición para G4 en §11.2–11.4.

Corte de datos: todos los resultados de esta sección provienen de corridas con `code_version` posterior al 2026-09-03 — fecha de corrección de dos bugs que afectaban directamente la percepción del VLM (canales R/B invertidos y marcadores de debug en la captura). Las corridas anteriores a esa fecha no son comparables para el brazo `slm` y no se usan en el análisis.

11.1 Resultados piloto (1 semilla, ilustrativo — pre-batch G4)

Estas corridas cumplen el criterio de arranque del diseño experimental (`success = True`, 0 colisiones) y sirven de referencia cualitativa antes del análisis estadístico de G4. Estrategia de desbloqueo: `deep_vlm` en todos los casos (default de producción).

Tier 0 — `minisim_clear` · MiniSim (crater.png)

Escenario: 3 waypoints en L (~191m), altitud -10m, ambiente despejado. Fecha: 2026-09-07.

Brazo	Éxito	Ciclos	Duración (s)	Distancia (m)	Colisiones	Invoc. SLM	Deliberación	Fallback SLM	Deadlocks
<code>slm</code>	✓	1151	235	191.7	0	80	6.95%	1.25%	1
<code>fsm</code>	✓	1031	208	224.4	0	—	—	—	2
<code>reactive</code>	✓	414	84	183.6	0	—	—	—	0

Dist. mín. al obstáculo: `slm` 9.74m · `fsm` 6.38m · `reactive` 9.06m

Observación: En un ambiente despejado la diferencia entre brazos es de velocidad pura. `reactive` completa en 84s (sin deliberar); `slm` tarda 2.8× más debido a las 80 invocaciones al VLM, aunque el trayecto real es el más corto (191.7m vs 224.4m del `fsm`).

Tier 1 — `townsim_clear` · TownSim (townsim_calib.png)

Escenario: perímetro completo del complejo, 6 WPs, ~640m, altitud de tránsito -30m (sobre los edificios). Fecha: 2026-09-07.

Brazo	Éxito	Ciclos	Duración (s)	Distancia (m)	Colisiones	Invoc. SLM	Deliberación	Fallback SLM	Deadlocks
<code>slm</code>	✓	1337	311	632.3	0	11	0.82%	0%	3
<code>fsm</code>	✓	1537	347	652.3	0	—	—	—	5
<code>reactive</code>	✓	1196	266	639.0	0	—	—	—	0

Dist. mín. al obstáculo: `slm` 0.004m · `fsm` 7.89m · `reactive` 9.51m

Observación: La tasa de deliberación del `slm` bajó de 15.4% (corridas pre-fix de agosto) a 0.82% — reducción de ×19, atribuible al avance cauteloso durante la espera del VLM (elimina el bucle mecánico de baja confianza) y al fix de colores R/B (reduce falsas alarmas de percepción). Con sólo 11 invocaciones sobre 1337 ciclos, el brazo `slm` es el más rápido de los tres en este escenario. El `fsm` acumula 5 deadlocks (todos resueltos por escaneo profundo); el `reactive`, 0. La distancia mínima al obstáculo del `slm` (0.004m) es un artefacto del spawn conocido (ciclo inicial, colisión estática con el suelo), no una aproximación peligrosa en vuelo.

Tier 2 — `citysim_clear` · CitySim (citysim_calib.png)

Escenario: perímetro de una manzana del grid regular, 7 WPs, ~430m, altitud de tránsito -70m (climb-first: WP_1→WP_2 sube vertical puro antes de mover en horizontal). Fecha: 2026-09-07.

Brazo	Éxito	Ciclos	Duración (s)	Distancia (m)	Colisiones	Invoc. SLM	Deliberación	Fallback SLM	Deadlocks
<code>slm</code>	✓	628	174	427.2	0	5	0.80%	0%	0

Brazo	Éxito	Ciclos	Duración (s)	Distancia (m)	Colisiones	Invoc. SLM	Deliberación	Fallback SLM	Deadlocks
fsm	✓	641	182	447.2	0	—	—	—	0
reactive	✓	541	159	431.2	0	—	—	—	0

Dist. mín. al obstáculo: `slm` 0.77m · `fsm` 11.0m · `reactive` 9.43m

Observación: Los tres brazos completan el perímetro sin colisiones ni deadlocks — el escenario base a -70m está por encima de la línea de tejados de CitySim. La arquitectura climb-first (near_vertical fix, 2026-09-07) evita el problema de la primera corrida (z=-50m, rampa diagonal que atravesaba edificios). Con 0 deadlocks en los tres brazos, `citysim_clear` confirma el mismo rol que `minisim_clear` y `townsim_clear`: escenario de control a altitud de tránsito libre, cota inferior para el análisis del brazo `slm` en Tier 2. La dist. mín. del `slm` (0.77m) es probable artefacto del segmento de climb cerca del spawn; no se registró ninguna evasión activa durante el vuelo horizontal.

11.2 Tabla de resultados por brazo y escenario

Datos de corridas piloto (seed=1, `deep_vlm`). La columna "DistMin" es el valor observado en la corrida única; el percentil p5 se calculará sobre ≥5 semillas en el batch G4. Las filas marcadas **[pendiente G4]** corresponden a condiciones no corridas aún.

Escenarios definitivos G4: `minisim_clear` (Tier 0), `townsim_ini` (Tier 1 con obstáculos reales) / `townsim_calib_cruce_frontal` (T-CALIB-2), `citysim_clear` (Tier 2 base) / `citymap_a` (Tier 2 con obstáculos — pendiente construcción). Las filas de `townsim_clear` corresponden al escenario base (sin obstáculos a altitud de tránsito), equivalente al rol de `minisim_clear` en Tier 0.

Brazo	Tier / Escenario	Estrategia Atasco	Éxito (1 sem.)	Col./km	DistMin (m)	Tiempo (s)	Deliberación	Fallback SLM	Res. Atasco VLM
<code>slm</code>	Tier 0 (<code>minisim_clear</code>)	<code>deep_vlm</code>	1/1 ✓	0	9.74	235	6.95%	1.25%	100% (1/1)
<code>slm</code>	Tier 0 (<code>minisim_clear</code>)	<code>blind</code>	—	—	—	—	—	—	—
<code>slm</code>	Tier 1 (<code>townsim_clear</code>)	<code>deep_vlm</code>	1/1 ✓	0	0.004*	311	0.82%	0%	100% (3/3)
<code>slm</code>	Tier 1 (<code>townsim_clear</code>)	<code>blind</code>	—	—	—	—	—	—	—
<code>slm</code>	Tier 2 (<code>citysim_clear</code>)	<code>deep_vlm</code>	1/1 ✓	0	0.77†	174	0.80%	0%	— (0 deadlocks)
<code>fsm</code>	Tier 0 (<code>minisim_clear</code>)	<code>deep_vlm</code>	1/1 ✓	0	6.38	208	—	—	100% (2/2)
<code>fsm</code>	Tier 1 (<code>townsim_clear</code>)	<code>deep_vlm</code>	1/1 ✓	0	7.89	347	—	—	100% (5/5)
<code>fsm</code>	Tier 2 (<code>citysim_clear</code>)	<code>deep_vlm</code>	1/1 ✓	0	11.0	182	—	—	— (0 deadlocks)
<code>reactive</code>	Tier 0 (<code>minisim_clear</code>)	—	1/1 ✓	0	9.06	84	—	—	—
<code>reactive</code>	Tier 1 (<code>townsim_clear</code>)	—	1/1 ✓	0	9.51	266	—	—	—
<code>reactive</code>	Tier 2 (<code>citysim_clear</code>)	—	1/1 ✓	0	9.43	159	—	—	—

* DistMin Tier 1 `slm` = 0.004m: artefacto del ciclo de spawn (contacto estático con suelo), no aproximación en vuelo. † DistMin Tier 2 `slm` = 0.77m: probable artefacto del segmento climb-first cerca del spawn; sin evasión activa registrada en vuelo horizontal.

11.3 Observaciones preliminares y marco para el análisis estadístico G4

Nota: con 1 semilla por celda no es posible ejecutar pruebas de significancia estadística. Esta sección presenta las tendencias observadas en los datos piloto y el diseño del análisis que se realizará sobre el batch G4 (≥5 semillas por celda).

11.3.1 Tendencias en los datos piloto

Los tres escenarios ejecutados hasta la fecha son escenarios de **control a altitud de tránsito libre**: la ruta no cruza ningún obstáculo en la dirección de vuelo horizontal — los edificios quedan por debajo del plano de crucero (~30m en Tier 1, ~70m en Tier 2). En esta configuración, los datos piloto muestran un patrón consistente en los tres tiers:

Escenario	Tiempo <code>reactive</code>	Tiempo <code>sLm</code>	Ratio <code>sLm / reactive</code>	Tiempo <code>fsm</code>	Ratio <code>fsm / reactive</code>
Tier 0 <code>minisim_clear</code>	84s	235s	2.8×	208s	2.5×
Tier 1 <code>townsim_clear</code>	266s	311s	1.17×	347s	1.30×
Tier 2 <code>citysim_clear</code>	159s	174s	1.09×	182s	1.14×

El ratio `sLm / reactive` decrece con la longitud de la ruta: en Tier 0 (~191m), la latencia fija del VLM (~2.9s por invocación × 80 invocaciones) domina el tiempo total; en Tier 2 (~430m), con solo 5 invocaciones y avance cauteloso durante la espera, el overhead es marginal. Esta relación es consistente con la hipótesis de que el costo del brazo `sLm` no es lineal en la distancia, sino principalmente en la frecuencia de deliberación.

Una segunda tendencia es la diferencia en distancia recorrida:

Escenario	Distancia <code>sLm</code>	Distancia <code>fsm</code>	Δ
Tier 0	191.7m	224.4m	-32.7m (~15%)
Tier 1	632.3m	652.3m	-20.0m (~3%)
Tier 2	427.2m	447.2m	-20.0m (~5%)

En los tres tiers, el brazo `sLm` recorre menos distancia que el `fsm`. Con 1 semilla, esto puede ser varianza del punto de spawn; con ≥5 semillas, si el patrón se sostiene, indicaría que las pocas invocaciones del VLM en escenarios de control producen correcciones de rumbo que evitan desviaciones que la FSM sí acumula.

Los deadlocks observados son escasos (0–5 por corrida) y todos resueltos al 100% por `deep_vlm` en `sLm` y `fsm`. El brazo `reactive` no registra deadlocks en ningún tier, lo cual es esperado: su control reactivo puro no tiene el concepto de "objetivo pendiente" que genera atascos en los brazos deliberativos cuando la ruta directa está bloqueada.

11.3.2 Marco estadístico para el batch G4

Sobre ≥5 semillas independientes por celda factorial se ejecutará:

- Prueba U de Mann-Whitney** (bilateral) comparando cada par de brazos en el mismo escenario y estrategia de desbloqueo. Hipótesis nula: la distribución de la métrica principal no difiere entre brazos. Métrica primaria: tiempo de misión en corridas con `success=True`; métrica secundaria: `dist_min_m` como indicador de seguridad.
- Tamaño de efecto:** Cliff's Delta (δ) sobre el mismo par. Umbral de referencia: $\delta > 0.3$ como efecto "mediano" (interpretación de Romano et al., 2006). Un p-valor significativo con δ pequeño (< 0.1) indicaría una diferencia real pero de magnitud práctica despreciable.
- Corrección de Bonferroni** sobre las comparaciones múltiples por tier (3 pares de brazos × 2 estrategias × 3 escenarios = 18 pruebas); umbral ajustado $\alpha = 0.05/18 \approx 0.0028$.

Las comparaciones previstas son: `sLm` vs. `fsm`, `sLm` vs. `reactive`, y `fsm` vs. `reactive`, tanto en estrategia `deep_vlm` como `blind` donde aplique. La comparación `deep_vlm` vs. `blind` dentro del mismo brazo `sLm` cuantifica el aporte de H3 (escaneo profundo deliberativo frente al escape ciego).

11.4 Análisis por tipo de escenario

La pregunta central de la tesis — ¿aporta la deliberación contextual del VLM sobre la heurística rígida de la FSM? — requiere desagregarla por tipo de escenario, porque la respuesta esperada es distinta según el nivel de dificultad.

11.4.1 Tier 0 — Control sin obstáculos (minisim_clear)

El escenario de referencia más simple: 3 waypoints en L, ambiente despejado, sin ningún obstáculo en la trayectoria directa. El dato piloto muestra el costo puro del brazo `s1m` sin ningún beneficio compensatorio: 80 invocaciones al VLM sobre 1151 ciclos (6.95%) generan 235s de tiempo de misión frente a los 84s del brazo `reactive` (ratio 2.8×). El `fsm` (208s) ocupa una posición intermedia: también sufre del overhead de su lazo deliberativo en atascos (2 deadlocks), pero sin la latencia VLM por invocación.

La distancia mínima al obstáculo es comparable entre brazos (9.74m `s1m`, 9.06m `reactive`, 6.38m `fsm`), confirmando que en un entorno despejado ningún brazo es más seguro que otro — la ventaja del VLM no tiene dónde manifestarse. Este resultado es el esperado por diseño: `minisim_clear` existe para establecer la cota inferior de rendimiento del sistema y cuantificar el overhead puro del brazo `s1m` en ausencia de beneficio.

Predicción G4: la diferencia de velocidad entre `reactive` y `s1m` debería sostenerse con alta significancia estadística (Cliff's δ cercano a 1.0 en favor de `reactive`); la comparación `s1m` vs. `fsm` puede ser más variable porque depende de cuántos deadlocks acumule la FSM por corrida.

11.4.2 Tier 1 — Perímetro urbano con vegetación (townsim_clear)

El escenario `townsim_clear` para Tier 1: vuela el perímetro a -30m (sobre la línea de tejados), sin obstáculos en la trayectoria de crucero. A diferencia de Tier 0, la longitud de la ruta (~640m vs. ~191m) diluye el overhead por invocación, y el brazo `s1m` resulta el más rápido de los tres en el piloto (311s vs. 347s `fsm` vs. 266s `reactive`). La tasa de deliberación de 0.82% (11 invocaciones / 1337 ciclos) representa una reducción de ×19 respecto a las corridas pre-fix de agosto (15.4%), directamente atribuible al avance cauteloso durante la espera del VLM y al fix de canales de color que reduce las falsas alarmas.

El dato más significativo de Tier 1 piloto no es la velocidad sino los deadlocks: el `fsm` acumula 5 (todos resueltos por escaneo profundo `deep_vlm`); el `s1m`, 3; el `reactive`, 0. Con 1 semilla, este patrón es indicativo pero no concluyente — puede reflejar diferencias en la gestión de atascos entre brazos, o simplemente la varianza de una única semilla.

El escenario de interés real para Tier 1 es `townsim_ini`: un recorrido que cruza el interior del complejo, con corredores vegetados y fachadas que obstruyen la trayectoria directa. Es allí donde el escaneo deliberativo profundo (`deep_vlm`) tiene un caso de uso genuino frente al escape ciego (`blind`): la FSM y el `reactive` deben bordear obstáculos por heurística, mientras el `s1m` puede consultar al VLM para elegir el corredor. Los datos de `townsim_clear` solo establecen la cota de partida.

11.4.3 Tier 2 — Entorno urbano denso (citysim_clear)

El escenario base `citysim_clear` funciona como el tercer escenario de control: los tres brazos completan el perímetro en ~159–182s, sin colisiones ni deadlocks, con tiempos comparables entre sí (ratio `s1m` / `reactive` = 1.09×). A -70m, el dron vuela por encima de la línea de tejados de CitySim; la deliberación del VLM no tiene obstáculo real que analizar.

Tres observaciones son relevantes para el análisis posterior:

1. **fsm conservador, reactive agresivo:** el `fsm` registra la mayor distancia mínima al obstáculo (11.0m) y el mayor tiempo (182s); el `reactive` el menor (9.43m, 159s). El `s1m` queda entre ambos (0.77m en climb inicial, 174s). En un entorno donde no hay obstáculos activos, la heurística conservadora de la FSM penaliza velocidad sin ganar seguridad.
2. **Altitud como variable crítica de diseño:** la primera corrida a z=-50m falló (success=False, colisión) porque la altitud insuficiente hacía que la ruta de climb desde el spawn atravesara edificios. El fix de near_vertical (2026-09-07) permite el patrón climb-first, pero la variable determinante fue la elección de -70m > altura de tejados.
3. **Cero deadlocks en los tres brazos:** confirma que la ruta de crucero no presenta obstrucciones reales a -70m. Cualquier deadlock que aparezca en `citymap_clear` — donde la ruta sí cruza corredores entre edificios — será atribuible a la geometría del escenario, no a artefactos de la altitud.

El escenario `citymap_clear` es el experimento inicial de Tier 2: un recorrido que atraviesa corredores angostos entre edificios altos, donde ni el `reactive` ni el `fsm` tienen información semántica para elegir entre dos calles de ancho similar. La hipótesis es que el brazo `s1m`, al consultar al VLM con una imagen aérea del corredor, podrá elegir la ruta más despejada con mayor consistencia que una heurística basada en distancia pura o en flujo óptico. El resultado negativo también es válido: si el VLM no aporta información útil en un entorno de alta textura urbana uniforme, es un hallazgo de diseño relevante para el capítulo 09.

12. Conclusiones

Estado: pendiente del capítulo de resultados (cap. 11), que a su vez depende de la corrida experimental comparativa descrita en el capítulo 10.

12.1 Retomar el hilo conductor

Cerrar aquí el argumento declarado en la introducción (§1.4) y desarrollado con evidencia medida en el capítulo 9: en un sistema de control donde un modelo de lenguaje consume descripciones de escena, el error más caro está en la interfaz que lo alimenta, no en el razonamiento del modelo, y las instancias documentadas de ese patrón —desincronización de esquemas de percepción y campos nulos, desalineación en secuencias temporales, saturación por descalibración de escala diferencial, y descarte silencioso en grafos de estado— se detectaron auditando formalmente los contratos de datos y la propagación de estado, nunca observando únicamente el comportamiento superficial del vuelo.

12.2 Respuesta a la pregunta de investigación

¿Aporta el SLM sobre la FSM? Síntesis de §11.4, con los matices por tipo de escenario que surjan de la corrida experimental — evitar una respuesta única y agregada si los datos sostienen una respuesta más matizada por escenario.

12.3 Limitaciones

- Validación exclusivamente en simulación (AirSim); sin validación en hardware físico (Jetson Nano + dron real), que el plan de trabajo aprobado (`plan_tesis/plan-tesis.md`, §"Transferencia de los resultados obtenidos") sitúa como siguiente etapa fuera del alcance de este trabajo.
- Calibración del canal de ocupación de `ObstacleField` pendiente al cierre de este escrito (cap. 6, §6.3), con el canal de TTC como el único de los dos formalmente validado contra profundidad (cap. 7).
- Tamaño de muestra y generalización: a determinar según el número final de semillas y escenarios de la corrida de tesis (cap. 10).

12.4 Trabajo futuro

- Validación en hardware real: companion computer de bajo costo (Jetson Nano) conectada a un dron físico con cámara monocular a bordo, siguiendo el pipeline descrito en el plan de trabajo aprobado.
- Calibración del canal de ocupación contra profundidad, con el mismo protocolo de curva ROC aplicado al TTC (cap. 7), y validación de la derotación con giros de guiñada más agresivos (cap. 7, §7.4).
- Navegación en enjambre, sensores adicionales, y las demás líneas de extensión discutidas en el plan de trabajo aprobado (`plan_tesis/plan-tesis.md`, §"Transferencia de los resultados obtenidos").

13. Referencias

Bibliografía compilada a partir de dos fuentes del proyecto: la citada en el plan de trabajo aprobado(`plan_tesis/plan-tesis.md`) y la disponible como archivos (PDF/BibTeX) en `plan_tesis/bibliografia/` y en `informe/bibliografia/`. Se organiza en cuatro secciones según su origen, para que la revisión y depuración posterior (eliminar entradas no citadas en el texto final, resolver duplicados menores de formato) sea trazable. El formato sigue el usado en el plan de trabajo aprobado (APA, con DOI o URL cuando está disponible).

13.1 Bibliografía citada en el plan de trabajo aprobado

Alarm as estate agent's drone crashes into house - Property Industry Eye. (s. f.). Recuperado 29 de junio de 2025, de https://propertyindustryeye.com/alarm-as-estate-agents-drone-crashes-into-house/?utm_source=chatgpt.com

Aldao, E., González-de Santos, L. M., & González-Jorge, H. (2022). LiDAR Based Detect and Avoid System for UAV Navigation in UAM Corridors. *Drones*, 6(8). <https://doi.org/10.3390/drones6080185>

Aliane, N. (2024). A Survey of Open-Source UAV Autopilots. *Electronics*, 13(23). <https://doi.org/10.3390/electronics13234785>

Arnold, R., Yamaguchi, H., & Tanaka, T. (2018). Search and rescue with autonomous flying robots through behavior-based cooperative intelligence. *Journal of International Humanitarian Action*, 3. <https://doi.org/10.1186/s41018-018-0045-4>

Broggi, A., Bertozzi, M., Fascioli, A., Guarino, C., Guarino Lo Bianco, C., & Piazzi, A. (2000). The Argo Autonomous Vehicle's Vision And Control Systems. *Int. J. Intelligent. Control. Syst.*, 3.

Cangahuala, L. A., Campagnola, S., Bradley, B. K., Boone, D. R., Buffington, B. B., Ludwinski, J. M., Nandi, S., & Scott, C. J. (2025). Europa Clipper Mission Design, Mission Plan, and Navigation. *Space Science Reviews*, 221(1), 22. <https://doi.org/10.1007/s11214-025-01140-2>

Castelli, T., Sharghi, A., Harper, D., Trémeau, A., & Shah, M. (2016). Autonomous navigation for low-altitude UAVs in urban areas. *CoRR*, abs/1602.08141. <http://arxiv.org/abs/1602.08141>

Castro, W., Marcato Junior, J., Polidoro, C., Osco, L. P., Gonçalves, W., Rodrigues, L., Santos, M., Jank, L., Barrios, S., Valle, C., Simeão, R., Carromeu, C., Silveira, E., Jorge, L. A. de C., & Matsubara, E. (2020). Deep Learning Applied to Phenotyping of Biomass in Forages with UAV-Based RGB Imagery. *Sensors*, 20(17). <https://doi.org/10.3390/s20174802>

Chahine, M., Hasani, R., Kao, P., Ray, A., Shubert, R., Lechner, M., Amini, A., & Rus, D. (2023). Robust flight navigation out of distribution with liquid neural networks. *Science Robotics*, 8(77). <https://doi.org/10.1126/scirobotics.adc8892>

Chen, F., Wu, Q., Tao, G., & Jiang, B. (2014). A Reconfiguration Control Scheme for a Quadrotor Helicopter via Combined Multiple Models. *International Journal of Advanced Robotic Systems*, 11(8), 122. <https://doi.org/10.5772/58833>

Chen, W., Shang, G., Hu, K., Zhou, C., Wang, X., Fang, G., & Ji, A. (2022). A Monocular-Visual SLAM System with Semantic and Optical-Flow Fusion for Indoor Dynamic Environments. *Micromachines*, 13(11). <https://doi.org/10.3390/mi13112006>

Collier, J., Trentini, M., Giesbrecht, J., Mcmanus, C., Furgale, P., Stenning, B., Barfoot, T., Se, S., Kotamraju, V., Jasiobedzki, P., Shang, L., Chan, B., Harmat, A., & Sharf, I. (2012, enero). Autonomous Navigation and Mapping in GPS-Denied Environments at Defence R&D Canada. *NATO Symposium SET 168: Navigation Sensor and Systems in GNSS Denied Environments*.

Da Silva, A., Fernández, S., Vidal, B., Sodre, H., Moraes, P., Peters, C., Barcelona, S., Sandin, V., Moraes, W., Mazondo, A., Macedo, B., Assunção, N., de Vargas, B., Kelbouscas, A., & Grando, R. (2024). *Implementación de Navegación en Plataforma Robótica Móvil Basada en ROS y Gazebo*. <http://arxiv.org/abs/2410.19972>

Dickmanns, E. D. (2024). Evolution of the "4-D Approach" to Dynamic Vision for Vehicles. *Electronics*, 13(20), 4133.

Drone Silo Inspections - Case Study - Horus Drones. (s. f.). Recuperado 29 de junio de 2025, de https://horusdrones.com/home/drone-insights/case-studies/drone-silo-inspection/?utm_source=chatgpt.com

Giacomossi Jr, L., Maximo, M., Sundelius, N., Funk, P., Brancalion, J. F. B., & Sohlberg, R. (2024). Cooperative Search and Rescue with Drone Swarm (pp. 381-393). https://doi.org/10.1007/978-3-031-39619-9_28

Goel, A., Tung, C., Hu, X., Thiruvathukal, G. K., Davis, J. C., & Lu, Y.-H. (2021). Efficient Computer Vision on Edge Devices with Pipeline-Parallel Hierarchical Neural Networks. *CoRR*, abs/2109.13356. <https://arxiv.org/abs/2109.13356>

Graf, M., Speidel, O., Ruof, J., & Dietmayer, K. (2021). *On-Road Motion Planning for Automated Vehicles at Ulm University*. <http://arxiv.org/abs/2012.04028>

Hoang, V. D., Nyboe, F. F., Malle, N. H., & Ebeid, E. (2024). Autonomous Overhead Powerline Recharging for Uninterrupted Drone Operations. <https://arxiv.org/abs/2403.06533>

Hu, J., Ren, Z., & Cheng, W. (2024). Finite State Machines-Based Path-Following Collaborative Computing Strategy for Emergency UAV Swarms. <https://arxiv.org/abs/2407.11531>

Hughes, M., & Engelbrecht, J. (2023). Autonomous navigation for multiple unmanned aerial vehicles (UAVs) in urban environments. *MATEC Web of Conferences*, 388. <https://doi.org/10.1051/mateconf/202338804011>

- Hwang, J.-J., Xu, R., Lin, H., Hung, W.-C., Ji, J., Choi, K., Huang, D., He, T., Covington, P., Sapp, B., Zhou, Y., Guo, J., Anguelov, D., & Tan, M. (2024). *EMMA: End-to-End Multimodal Model for Autonomous Driving*. <https://arxiv.org/abs/2410.23262>
- Infocampo. (s. f.). *Renova amplió su planta de procesamiento y construirá una nueva terminal en Timbués*. INFOCAMPO Noticias del campo en el momento justo. Recuperado 25 de junio de 2025, de <https://www.infocampo.com.ar/renova-amplio-su-planta-de-procesamiento-y-construira-una-nueva-terminal-en-timbues/>
- Kemeny, A., & Panerai, F. (2003). Evaluating perception in driving simulation experiments. *Trends in Cognitive Sciences*, 7(1), 31-37. [https://doi.org/10.1016/S1364-6613\(02\)00011-6](https://doi.org/10.1016/S1364-6613(02)00011-6)
- Koubaa, A., Allouch, A., Alajlan, M., Javed, Y., Belghith, A., & Khalgui, M. (2019). Micro Air Vehicle Link (MAVLink) in a Nutshell: A Survey. *CoRR*, abs/1906.10641. <http://arxiv.org/abs/1906.10641>
- Langåker, H.-A., Kjerkeit, H., Syversen, C. L., Moore, R. J. D., Holhjem, Ø. H., Jensen, I., Morrison, A., Transeth, A. A., Kvien, O., Berg, G., Olsen, T. A., Hatlestad, A., Negård, T., Broch, R., & Johnsen, J. E. (2021). An autonomous drone-based system for inspection of electrical substations. *International Journal of Advanced Robotic Systems*, 18(2), 17298814211002972. <https://doi.org/10.1177/17298814211002973>
- Li, Q., Yu, W., & Jiang, S. (2022). *Optimized Views Photogrammetry: Precision Analysis and A Large-scale Case Study in Qingdao*. <https://arxiv.org/abs/2206.12216>
- Madridano Carrasco, Á. (2020). *Arquitectura de software para navegación autónoma y coordinada de enjambres de drones en labores de lucha contra incendios forestales y urbanos* [Tesis doctoral, Universidad Carlos III de Madrid]. <https://hdl.handle.net/10016/32463>
- Marazzato, R., & Sparavigna, A. C. (2015). Retinex filtering of foggy images: generation of a bulk set with selection and ranking. *CoRR*, abs/1509.08715. <http://arxiv.org/abs/1509.08715>
- Martin, P. G., Scott, T. B., Payton, O. D., & Fardoulis, J. S. (2017). *High-Resolution Aerial Radiation Mapping for Nuclear Decontamination and Decommissioning-17371*.
- Matej, J. (2016). *Simulation of vehicle in Unreal Game Engine using equations of motion, Runge-Kutta and line tracing methods to solve motion and scan Unreal Engine terrain under wheel*. Publishing House at the Technical University in Zvolen. <https://doi.org/10.5281/zenodo.55806>
- Miranda, V., Rezende, A., Rocha, T., Azpúrua, H., Pimenta, L., & Freitas, G. (2021). Autonomous Navigation System for a Delivery Drone. *Journal of Control, Automation and Electrical Systems*, 33. <https://doi.org/10.1007/s40313-021-00828-4>
- Mur-Artal, R., Montiel, J. M. M., & Tardós, J. D. (2015). ORB-SLAM: a Versatile and Accurate Monocular SLAM System. *CoRR*, abs/1502.00956. <http://arxiv.org/abs/1502.00956>
- Nguyen, D. D., Rohacs, J., & Rohacs, D. (2021). Autonomous Flight Trajectory Control System for Drones in Smart City Traffic Management. *ISPRS International Journal of Geo-Information*, 10(5). <https://doi.org/10.3390/ijgi10050338>
- Nothdurft, T., Hecker, P., Ohl, S., Saust, F., Maurer, M., Reschka, A., & Rüdiger Böhmer, J. (2011). *Stadtpilot: First Fully Autonomous Test Drives in Urban Traffic*. https://www.tu-braunschweig.de/fileadmin/Redaktionsgruppen/Institute_Fakultaet_5/IFR/Dateien_EFS/Publikationen/Stadtpilot_FirstFullyAutonomous.pdf
- NVIDIA. (2025). *Jetson Nano Brings the Power of Modern AI to Edge Devices*. <https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-nano/product-development/>
- Open Source Autopilot for Drones - PX4 Autopilot. (s. f.). Recuperado 29 de junio de 2025, de <https://px4.io/>
- Pestana Puerta, J. (2017). *Vision-Based Autonomous Navigation of Multirotor Micro Aerial Vehicles* [PhD Thesis, Universidad Politécnica de Madrid]. <https://doi.org/10.20868/UPM.thesis.47726>
- Pomerleau, D. A. (1988). ALVINN: An Autonomous Land Vehicle in a Neural Network. En D. Touretzky (Ed.), *Advances in Neural Information Processing Systems* (Vol. 1). Morgan-Kaufmann. https://proceedings.neurips.cc/paper_files/paper/1988/file/812b4ba287f5ee0bc9d43bbf5bbe87fb-Paper.pdf
- Redmon, J., Divvala, S. K., Girshick, R. B., & Farhadi, A. (2015). You Only Look Once: Unified, Real-Time Object Detection. *CoRR*, abs/1506.02640. <http://arxiv.org/abs/1506.02640>
- Resolución ANAC - Reglamento de Vehículos Aéreos No tripulados, Pub. L. No. 880/2019 (2019). <https://www.argentina.gob.ar/normativa/nacional/resoluci%C3%B3n-880-2019-333259>
- Samy, M., Amer, K., Shaker, M., & ElHelw, M. (2019). *Drone Path-Following in GPS-Denied Environments using Convolutional Networks*. <https://arxiv.org/abs/1905.01658>
- Shah, S., Dey, D., Lovett, C., & Kapoor, A. (2017). *AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles*. <http://arxiv.org/abs/1705.05065>
- Shin, Y.-S., & Kim, A. (2019). Sparse Depth Enhanced Direct Thermal-infrared SLAM Beyond the Visible Spectrum. *IEEE Robotics and Automation Letters*, 4(3), 2918-2925. <https://arxiv.org/abs/1902.10892>
- Song, C., Guo, X., & Sui, J. (2025). Improved Model Predictive Control Algorithm for the Path Tracking Control of Ship Autonomous Berthing. *Journal of Marine Science and Engineering*, 13(7). <https://doi.org/10.3390/jmse13071273>

- Thorpe, C., Hebert, M. H., Kanade, T., & Shafer, S. A. (1988). Vision and navigation for the Carnegie-Mellon Navlab. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 10(3), 362-373. <https://doi.org/10.1109/34.3900>
- Turco, L., Zhao, J., Xu, Y., & Tsourdos, A. (2024). A Study on Co-simulation Digital Twin with MATLAB and AirSim for Future Advanced Air Mobility. *2024 IEEE Aerospace Conference*, 1-18. <https://doi.org/10.1109/AERO58975.2024.10521333>
- UMILES GROUP. (2024, diciembre 20). *LiDAR y Fotogrametría: Las armas secretas de los drones en la topografía moderna*. <https://umilesgroup.com/drones-lidar-y-fotogrametria-cambiando-la-topografia-moderna/>
- Vipparla, C., Krock, T., Nouduri, K., Fraser, J., AliAkbarpour, H., Sagan, V., Cheng, J.-R. C., & Kannappan, P. (2024). Fusion of Visible and Infrared Aerial Images from Uncalibrated Sensors Using Wavelet Decomposition and Deep Learning. *Sensors*, 24(24). <https://doi.org/10.3390/s24248217>
- Wahba, G. (1965). A Least Squares Estimate of Satellite Attitude. *SIAM Review*, 7(3), 409. <https://doi.org/10.1137/1007077>
- Shi, Y., Li, X., & Zhang, J. (2024). Real-time obstacle avoidance for UAVs using lightweight deep learning. *IEEE Transactions on Industrial Electronics*, 71(8), 9123-9132.
- Vera-Yanez, R., Maza, I., & Ollero, A. (2024). Optical-Flow-Based Air-to-Air Detection of Airborne Obstacles for Autonomous Drones. *Drones*, 8(12), 705. <https://doi.org/10.3390/drones8120705>
- Wang, H., Zhang, X., & Liu, C. (2023). Urban infrastructure inspection using autonomous UAVs: A review. *Automation in Construction*, 154, 105020.
- World Health Organization. (2023). *Global status report on road safety 2023*. WHO Guidelines Approved by the Guidelines Review Committee. World Health Organization.
- Yang, Y., & Wei, P. (2024). Autonomous eVTOL trajectory optimization for urban air mobility. *Transportation Research Part C: Emerging Technologies*, 158, 104423.
- Zhang, T., & Wu, Q. (2023). Vision-based power line inspection with UAVs: Methods and challenges. *IEEE Geoscience and Remote Sensing Magazine*, 11(2), 78-95.
- Zhao, K., Liu, Z., & Chen, X. (2024). Deep reinforcement learning for UAV autonomous navigation in GPS-denied environments. *IEEE Transactions on Intelligent Transportation Systems*, 25(3), 2890-2902.
- Zhou, Y., Bautista, J., Yao, W., & de Marina, H. G. (2025). *Inverse Kinematics on Guiding Vector Fields for Robot Path Following*. <https://arxiv.org/abs/2502.17313>
- Zhu, Y., Moniz, J. R. A., Bhargava, S., Lu, J., Piraviperumal, D., Li, S., Zhang, Y., Yu, H., & Tseng, B.-H. (2024). *Can Large Language Models Understand Context?*. <https://arxiv.org/abs/2402.00858>

13.2 Bibliografía adicional en `plan_tesis/bibliografia/` (con archivo BibTeX, no citada arriba)

- AlMahamid, F., & Grolinger, K. (2022). Autonomous unmanned aerial vehicle navigation using reinforcement learning: A systematic review. *Engineering Applications of Artificial Intelligence*, 115, 105321.
- Baheri, A. (2020). *Safe Reinforcement Learning with Mixture Density Network: A Case Study in Autonomous Highway Driving*. <https://arxiv.org/abs/2007.01698>
- Bouhamed, O., Ghazzai, H., Besbes, H., & Massoud, Y. (2020). *Autonomous UAV Navigation: A DDPG-based Deep Reinforcement Learning Approach*. *CoRR*, abs/2003.10923. <https://arxiv.org/abs/2003.10923>
- Fabra, F., Zamora, W., Sangüesa, J., Calafate, C. T., Cano, J. C., & Manzoni, P. (2019). A Distributed Approach for Collision Avoidance between Multirotor UAVs Following Planned Missions. *Sensors*, 19(10), 2404. <https://doi.org/10.3390/s19102404>
- Grando, R. B., Pinheiro, P. M., Bortoluzzi, N. P., da Silva, C. B., Zauk, O. F., Piñeiro, M. O., Aoki, V. M., Kelbouscas, A. L., Lima, Y. B., Drews, P. L., & Neto, A. A. (2020). Visual-based Autonomous Unmanned Aerial Vehicle for Inspection in Indoor Environments. *2020 Latin American Robotics Symposium (LARS) / 2020 Brazilian Symposium on Robotics (SBR) / 2020 Workshop on Robotics in Education (WRE)*, 1-6. <https://doi.org/10.1109/LARS/SBR/WRE51543.2020.9307024>
- Jwo, D. J., Biswal, A., & Mir, I. A. (2023). Artificial Neural Networks for Navigation Systems: A Review of Recent Research. *Applied Sciences*, 13(7), 4475. <https://doi.org/10.3390/app13074475>
- Li, M., Li, J., Cao, Y., & Chen, G. (2024). A Dynamic Visual SLAM System Incorporating Object Tracking for UAVs. *Drones*, 8(6), 222. <https://doi.org/10.3390/drones8060222>
- Li, X., & Xiang, L. (2024). *Photorealistic Robotic Simulation using Unreal Engine 5 for Agricultural Applications*. <https://arxiv.org/abs/2405.18551>
- Polvara, R., Patacchiola, M., Sharma, S., Wan, J., Manning, A., Sutton, R., & Cangelosi, A. (2017). *Autonomous Quadrotor Landing using Deep Reinforcement Learning*. <https://doi.org/10.48550/arXiv.1709.03339>
- Saxena, P., Raghuvanshi, N., & Goveas, N. (2025). *UAV-VLN: End-to-End Vision Language guided Navigation for UAVs*. <https://arxiv.org/abs/2504.21432>

13.3 PDFs en `plan_tesis/bibliografia/` sin metadata bibliográfica estructurada

Sin archivo `.bib` asociado ni metadata completa embebida en el PDF; título tomado del nombre de archivo (convención "Autor - Título.pdf" ya usada en la carpeta). Año, autoría completa y venue de publicación quedan pendientes de completar durante la depuración.

- Comas (s.f., PDF fechado ~2020). *Modelado e implementación de sistemas de navegación aérea autónoma*. [Ver `plan_tesis/bibliografia/Comas - Modelado e implementación de sistemas de navegación aérea autónoma.pdf`; sin metadata de autoría/venue.]
- Presidencia de la Nación Argentina. Decreto 663/2024 — Aviación No Tripulada. [Ver `plan_tesis/bibliografia/Decreto 663-2024 Presidencia - Aviacion No tripulada.pdf`.]
- Nielsen, S. M. (2022). *A Visually Realistic Simulator for Autonomous eVTOL Aircraft*. [Ver `plan_tesis/bibliografia/`; venue de publicación pendiente de confirmar.]
- Ramezani, M. (s.f., PDF fechado ~2023). *UAV Path Planning Employing MPC Reinforcement*. [Ver `plan_tesis/bibliografia/Ramezani - UAV Path Planning Employing MPC Reinforcement.pdf`; venue pendiente.]
- Revista Seguros AAPAS. (2025). *Drones: todo lo que necesitas saber sobre regulaciones y seguros*.
- Sánchez Rodríguez (s.f., PDF fechado ~2019). *Navegación autónoma de un dron hexacóptero con visión estereoscópica*. [Autoría completa pendiente de confirmar.]
- Zhen (s.f., PDF fechado ~2025). *Training-efficient deep reinforcement learning for autonomous driving*. [Metadata de autor del PDF no confiable (aparenta ser el editor del documento, no el autor del paper); verificar durante la depuración.]

13.4 Bibliografía adicional en `informe/bibliografia/` (no incluida en el plan de trabajo aprobado)

- Badrloo, S., & Varshosaz, M. (2017). Monocular vision based obstacle detection. *International Journal of Earth Observation and Geomatics Engineering*, 1(2), 122-130.
- Chen, X., Kundu, K., Zhang, Z., Ma, H., Fidler, S., & Urtasun, R. (2016). Monocular 3D Object Detection for Autonomous Driving. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Geng, S., Cooper, H., Moskal, M., Jenkins, S., Berman, J., Ranchin, N., West, R., Horvitz, E., & Nori, H. (2025). *Generating Structured Outputs from Language Models: Benchmark and Studies*. <https://arxiv.org/abs/2501.10868>
- Jansen, W., Verreycken, E., Schenck, A., Blanquart, J.-E., Verhulst, C., Huebel, N., & Steckel, J. (2023). *Cosys-AirSim: A Real-Time Simulation Framework Expanded for Complex Industrial Applications*. <https://arxiv.org/abs/2303.13381>
- Kaneko, N., Yoshida, T., & Sumi, K. (2017). Fast Obstacle Detection for Monocular Autonomous Mobile Robots. *SICE Journal of Control, Measurement, and System Integration*, 10(5), 370-377.
- Molineros, J., Cheng, S. Y., Owechko, Y., Levi, D., & Zhang, W. (2012). Monocular Rear-View Obstacle Detection Using Residual Flow. *Computer Vision – ECCV 2012 Workshops and Demonstrations*, LNCS 7584. Springer.
- Al-Kaff, A., García, F., Martín, D., De La Escalera, A., & Armingol, J. M. (2017). Obstacle Detection and Avoidance System Based on Monocular Camera and Size Expansion Algorithm for UAVs. *Sensors*, 17(5), 1061.
- Raspanti, F., Ozcelebi, T., & Holenderski, M. (2025). Grammar-Constrained Decoding Makes Large Language Models Better Logical Parsers. *ACL 2025 Industry Track*.
- Rill, R.-A., & Faragó, K. B. (2021). Collision Avoidance Using Deep Learning-Based Monocular Vision. *SN Computer Science*, 2, 375. <https://doi.org/10.1007/s42979-021-00759-6>
- Shi, S., Ni, J., Kong, X., Zhu, H., Zhan, J., Sun, Q., & Xu, Y. (2024). An Obstacle Detection Method Based on Longitudinal Active Vision. *Sensors*, 24(14), 4407.
- Vera-Yanez, D., Pereira, A., Rodrigues, N., Molina, J. P., García, A. S., & Fernández-Caballero, A. (2024). Optical Flow-Based Obstacle Detection for Mid-Air Collision Avoidance. *Sensors*, 24(9), 3016.
- Zhou, Z., Liao, Y., Wang, B., Wang, M., Fu, M., & Hu, Z. (2026). Near obstacles detection by inverse perspective mapping of AVM for intelligent vehicles. *PLOS ONE*, 21(1), e0336851.

15 Anexos

15.1 A1. Exploración SLM / GGUF

Nota de ubicación: este documento es material exploratorio de la etapa de planificación (selección de modelos SLM en formato GGUF, técnicas de gramática restringida, evaluación del nivel de innovación de la arquitectura). No es un capítulo de la tesis; se conserva como anexo de referencia para la sección de selección de modelo en `08-DECISIONES-SLM.md`. Reubicado desde `informe/01-INTRO.md` el 2026-08-25.

15.1.1 Limitaciones de hardware

Para una configuración compuesta por una RTX 5060 con 8 GB de VRAM, en la que es necesario compartir recursos de GPU con una simulación de Unreal Engine 5.5, la opción más realista consiste en emplear un *Small Language Model (SLM)*. Se requiere un modelo *instruct-tuned*, capaz de seguir instrucciones de manera fiable y, al mismo tiempo, producir una salida con **gramática muy restringida o limitada**; es decir, un modelo que genere texto estrictamente conforme a un formato, esquema o gramática predefinida (por ejemplo, JSON siempre válido, pares clave-valor específicos o una sintaxis personalizada mínima sin variación libre).

Esta configuración resulta adecuada para una solución local basada en MCP (Model Context Protocol), el estándar abierto (propuesto por Anthropic a partir de 2024) que permite a los modelos de lenguaje conectarse con herramientas y datos externos mediante un protocolo uniforme. Un cliente MCP, un LLM local y servidores MCP locales (por ejemplo, para acceso a archivos, ejecución de código o consultas relacionadas con Unreal Engine) pueden ejecutarse completamente sin conexión.

Razones por las que un SLM con salida restringida se ajusta al escenario

- **Presupuesto de VRAM:** una simulación de UE5 probablemente utiliza entre 4 y 6 GB, lo que deja aproximadamente 2–4 GB disponibles para la inferencia del LLM (considerando el overhead de la KV cache).
- **Gramática restringida:** reduce alucinaciones y verbosidad, generando una salida pequeña, rápida y fácil de interpretar (crítico para llamadas a herramientas MCP o respuestas estructuradas).
- **Ejecución local:** elimina dependencias de la nube y permite baja latencia en la integración con la simulación.

Modelos recomendados (febrero 2026, en formato GGUF para llama.cpp / LM Studio)

Se recomienda centrarse en modelos instruct de **3B a 8B** parámetros, ya que ofrecen la capacidad suficiente para seguir instrucciones y manejar patrones de uso de herramientas en 2026. Las cuantizaciones Q5_K_M o Q4_K_M permiten ajustar el modelo a ~2–4.5 GB de VRAM manteniendo una calidad elevada.

Modelo	Tamaño (quant)	VRAM aprox. (full offload)	Fortalezas	Razones para uso con gramática restringida	Disponibilidad
Qwen3-4B-Instruct / Qwen3-7B-Instruct	~3–5 GB	~2.5–4 GB	Razonamiento sólido, seguimiento de instrucciones, function-calling	Cumplimiento fiable de formatos; variantes 2026 con buen soporte JSON	Qwen/Qwen3-4B-Inst-struct-GGUF
Phi-4-mini-instruct	~3–4 GB	~2–3.5 GB	Ajustado por Microsoft para datos sintéticos de alta calidad; muy eficaz en tareas estructuradas	Muy alta adherencia a esquemas y salida de baja variación	microsoft/Phi-4-mini-instruct-GGUF
SmolLM3-3B-Instruct	~2–3 GB	~1.8–3 GB	Modelo compacto con rendimiento superior al de muchos 4–7B	Fácil de forzar en formatos rígidos mediante prompt	HuggingFaceTB/SmolLM3-3B-GGUF
Gemma-3-4B-IT	~3 GB	~2.5 GB	Ajustado por Google; fuerte rendimiento textual	Buena salida estructurada y soporte de function-calling	google/gemma-3-4b-it-GGUF
Ministral-3-3B-Instruct	~2.5 GB	~2 GB	Modelo compacto optimizado para edge	Diseñado para uso restringido; seguimiento estable de formatos	mistralai/Ministral-3-3B-Instruct-GGUF

La opción preferente suele ser *Phi-4-mini-instruct*, mientras que *Qwen3-4B/7B* puede utilizarse cuando se dispone de aproximadamente 1 GB adicional de VRAM.

Métodos para imponer “gramática limitada” o formato de salida estricto

Los siguientes métodos, compatibles con backends locales como llama.cpp, LM Studio u Ollama, permiten controlar estrictamente la estructura de la salida:

1. **Prompt del sistema con instrucciones estrictas** (método más simple, sin costo computacional significativo)
Ejemplo: Eres un respondedor MCP estricto. Produce ÚNICAMENTE JSON válido que coincida con este esquema. Sin explicaciones, sin texto adicional, sin markdown. Esquema: { "tool_call": { "name": str, "args": dict }, "response": str o null } Si no se necesita herramienta, usar tool_call = null. Escapar correctamente todas las cadenas.

2. Gramáticas / GBNF en llama.cpp (confiabilidad alta)

- Definir una gramática libre de contexto mínima (GBNF) obliga al modelo a ajustarse exactamente al formato requerido.
- Compatible nativamente con llama.cpp y expuesto en diversas interfaces como LM Studio.
- Suele reducir la velocidad de generación en un 10–30%, pero garantiza validez estructural completa.

3. Outlines / Guidance / Ilguidance (métodos avanzados)

- Permiten imponer esquemas JSON, expresiones regulares o gramáticas personalizadas a nivel de token.
- Garantizan salida estructurada incluso en modelos pequeños.

Para MCP, estos métodos resultan especialmente relevantes, ya que numerosas implementaciones locales (por ejemplo, clientes y servidores MCP publicados en GitHub) requieren formatos fijos como JSON o XML estilo Anthropic.

Procedimiento práctico de configuración

1. Instalación de LM Studio

- Descargar un modelo GGUF entre los recomendados (desde el buscador integrado o Hugging Face).
- Configurar entre 20 y 35 capas en GPU, ajustando según el comportamiento de Unreal Engine.
- Activar el modo de gramática si está disponible o utilizar un prompt de sistema personalizado.

2. Implementación basada en MCP

- Iniciar con servidores MCP locales básicos (acceso a archivos, shell, etc.).
- Utilizar un cliente MCP compatible con modelos locales (Ollama con plugins MCP o implementaciones Python como mcp-agent).
- Proporcionar las descripciones de herramientas MCP mediante el prompt del sistema.

3. Integración con Unreal Engine 5

- Implementar un servidor MCP que lea/escriba logs, datos de Blueprints o estados de simulación.
- El modelo produce comandos estructurados que pueden ser interpretados y aplicados por la aplicación anfitriona.

15.1.2 Sobre la innovación de la solución

De manera general, la *arquitectura en bucle* descrita —un lazo cerrado simple y compacto donde:

1. AirSim proporciona el estado actual (posición, velocidad, datos de sensores, etc.),
2. Un SLM local y de pequeño tamaño procesa dicha información junto con el objetivo o las instrucciones,
3. Se genera un comando estructurado (por ejemplo, un JSON con parámetros de movimiento, guiñada o empuje),
4. El comando se ejecuta mediante la API de AirSim,
5. El proceso se repite a una frecuencia utilizable (por ejemplo, 2–20 Hz),

— resulta *sólida, práctica y adecuada para las restricciones planteadas* (RTX 5060 con 8 GB de VRAM compartida con una simulación en UE 5.5, y uso de modelos pequeños con gramática o salida restringida).

No obstante, en términos de *verdadera innovación* dentro del panorama 2025–2026 sobre control de drones/UAV impulsado por LLMs, dicha arquitectura *ya no se considera especialmente novedosa*. Se trata más bien de uno de los enfoques base que se han estandarizado tanto en investigación como en prototipos aplicados.

Razones por las que ya no se considera una solución de vanguardia

A partir de artículos, tesis, proyectos y experimentos recientes (principalmente entre 2025 y principios de 2026), se observa que:

- El *control en bucle cerrado mediante LLMs* para UAVs/drones en simuladores como AirSim se ha vuelto *muy común*. La mayoría de los trabajos emplea alguna variante del ciclo de retroalimentación.
 - Muchos lo denominan explícitamente “*closed-loop*” (por ejemplo, “LLM-driven closed-loop UAV operation”, “closed-loop reasoning/refinement”, “closed user-on-the-loop”).
 - La idea básica de alimentar el estado → LLM → comando → ejecutar → observar → repetir se ha demostrado repetidamente desde aproximadamente 2023–2024 y se generalizó en contextos UAV hacia 2025.
- La combinación *AirSim + LLM* se ha convertido en una opción *muy utilizada*, debido a que AirSim (basado en Unreal Engine) proporciona un entorno realista para probar física, visión y control de drones. Entre las variantes exploradas se encuentran:
 - Bucles de generación y ejecución de código.
 - Transformación semántica de observaciones (telemetría → descripciones textuales).
 - Configuraciones multi-agente con RAG.

- Tuberías voz-a-comando.
- Evaluación de ataques adversarios orientados a medir la robustez de LLMs en lazo cerrado.
- Las propuestas consideradas “innovadoras” en 2025–2026 suelen ir *más allá* del simple bucle estado → SLM → comando, e incluyen:
 - Diseños *dual-LLM* (uno genera código/acciones y otro evalúa/refina dentro de la simulación antes de ejecutar).
 - *Mejoras semánticas* para traducir telemetría en descripciones más comprensibles para modelos pequeños.
 - Enfoques *multi-agente* o *jerárquicos* (planificador + ejecutor + crítico).
 - *Visión en el bucle* (uso de VLMs pequeños como MiniCPM-V para evitar obstáculos).
 - *Síntesis de código en tiempo real* con módulos de corrección o RAG adaptativo.
 - Inferencia en *edge* para reducir aún más la latencia.

En consecuencia, la arquitectura propuesta se corresponde con el patrón básico ampliamente difundido en investigación sobre robótica/UAV basada en LLMs.

Ámbitos en los que la propuesta sí presenta solidez y utilidad

- Se adecúa de forma precisa a las limitaciones del hardware disponible: un SLM pequeño + inferencia rápida mediante llama.cpp permiten mantener latencias suficientemente bajas para tasas de control significativas, mientras que arquitecturas más complejas superarían el límite práctico de 8 GB de VRAM compartida con UE.
- La imposición de *salida estrictamente estructurada* (por ejemplo, JSON mediante gramáticas GBNF) constituye un enfoque actual y recomendable, utilizado en numerosos trabajos posteriores a 2025 para reducir alucinaciones en señales de control.
- Conseguir una operación totalmente local/offline dentro de UE5.5 + AirSim sigue siendo un reto para usuarios no especializados; muchos ejemplos conocidos emplean LLMs en la nube o hardware de mayor capacidad.

Conclusión (evaluación general)

- **Nivel de innovación:** aproximadamente 3–4/10. Se trata de una solución práctica y bien formulada, pero no representa una arquitectura novedosa en el contexto de 2026. Se ajusta más al ámbito de *ingeniería sólida basada en patrones establecidos* que a propuestas conceptualmente nuevas.
- No obstante, una implementación exitosa (vuelo estable, navegación reactiva, baja tasa de fallos en la simulación) constituye un logro destacable, especialmente bajo restricciones de VRAM y con un entorno UE ejecutándose de forma simultánea.
- Si se busca añadir un componente más innovador, es posible incorporar una de las extensiones comunes en 2025–2026, como un resumen semántico del estado, un pequeño evaluador para autocorrección o algún canal de retroalimentación visual, siempre que el presupuesto de VRAM y velocidad lo permita.

REFERENCIA: (Large Language Model-Driven Closed-Loop UAV Operation with Semantic Observations)[https://arxiv.org/html/2507.01930v1]

15.1.3 Sobre innovar sobre una arquitectura ya extensamente utilizada

Para que un bucle de control de cuadricóptero en AirSim basado en un SLM resulte *genuinamente innovador* en 2026 —especialmente bajo la fuerte restricción de hardware impuesta por una RTX 5060 con 8 GB de VRAM compartida con una simulación en UE 5.5— es necesario ir más allá del patrón estándar “estado → prompt → salida estructurada → actuar → repetir”. Dicho patrón ya constituye la base común en prototipos de investigación actuales.

A continuación se presentan los *giros de procesamiento/software* más prometedores que pueden situar la propuesta en el terreno de una contribución novedosa, ordenados aproximadamente según viabilidad en el hardware disponible y posible impacto:

Técnica / Enfoque	Nivel de innovación (2026)	Impacto en VRAM / Velocidad (8GB compartidos)	Razón por la que puede ser novedosa en este contexto	Dificultad de implementación	Beneficio realista para la autonomía del dron
Compresión semántica del estado + alimentación en lenguaje natural	Alta	Baja (añade ~0.2–0.5 GB de KV cache)	La mayoría de los trabajos usa telemetría cruda; los SLM pequeños razonan mejor con descripciones ricas (“deriva 0.8 m/s hacia la izquierda, obstáculo rojo a 2.1 m, batería 67%”).	Media	20–50% de mejora en calidad de decisiones / menos choques
Autocorrección / refinamiento en bucle cerrado en una sola pasada	Muy alta	Media (prompts más largos)	Permite incluir un campo “crítico” en la salida: el modelo propone acción + evalúa riesgos en la misma inferencia. Pocas implementaciones en entornos de baja VRAM.	Media–Alta	Aumento notable de confiabilidad (80–95% de acciones válidas)
Speculative decoding / borradores asistidos (con cabeza ligera)	Alta	Baja–Media (si se usa estilo EAGLE)	Acelera la generación 1.8–3× sin pérdida de calidad. Muy poco explorado en drones con baja VRAM.	Media–Alta	2–3× más frecuencia de bucle (p. ej., 10–15 Hz en lugar de 3–5 Hz)
Híbrido reactivo + LLM (PID a sub-ms + LLM solo para nivel alto)	Media–Alta	Muy baja	El LLM se utiliza solo para metas/replanificación; la estabilización se delega a controladores reactivos. Reduce dependencia de latencia.	Baja–Media	Vuelo más estable; narrativa innovadora de “control híbrido confiable”

Técnica / Enfoque	Nivel de innovación (2026)	Impacto en VRAM / Velocidad (8GB compartidos)	Razón por la que puede ser novedosa en este contexto	Dificultad de implementación	Beneficio realista para la autonomía del dron
<i>Fine-tuning diminuto en tiempo real / cambio de adaptadores LoRA</i>	Muy alta	Media-Alta (100–300 MB por adaptador)	Conjunto de LoRAs pequeños (“evitación agresiva”, “crucero eficiente”, etc.) con carga dinámica. Pocas demostraciones en simulación de baja capacidad.	Alta	Adaptación gradual al entorno en UE → efecto de “aprendizaje”
<i>Chain-of-thought multi-etapa con muestreo en árbol / batching</i>	Media-Alta	Media	Se fuerzan 2–3 futuros posibles en una sola salida; una heurística rápida elige el más seguro. Reduce latencia efectiva.	Media	Mejor manejo de incertidumbre / ráfagas de viento en AirSim

“Combinación ganadora” más prometedora para innovación + hardware disponible

Se recomienda apuntar a la siguiente arquitectura, factible dentro del límite de 8 GB de VRAM compartidos:

- 1. **Base:** Phi-4-mini-Instruct o Qwen3-4B/7B en Q5_K_M GGUF (~2.5–4 GB cargados; offload parcial si fuera necesario).
- 2. **Giro central A:** Preprocesamiento mediante *compresión semántica*. Una función Python ligera convierte telemetría de AirSim + objetivo en una descripción vívida en lenguaje natural. Esto potencia significativamente la capacidad de razonamiento de un SLM pequeño.
- 3. **Giro central B:** Salida con *autocorrección forzada por gramática*:

```
{
  "proposed_action": {"thrust": 0.72, "yaw_delta": -12, "move_vector": [1.2, -0.4, 0.8]},
  "confidence": 0.89,
  "risks": ["batería baja → reducir velocidad si <30%"],
  "alternative_if_risk_high": {"thrust": 0.55, "hover": true}
}
```

La salida se analiza y se aplica la acción más segura o con mayor confianza. Una sola inferencia gestiona propuesta + crítica.

- 4. **Aceleración:** Activar *speculative decoding* si la versión de llama.cpp o LM Studio lo permite (implementaciones estilo EAGLE o Medusa; comunes en 2026). Puede reducir la latencia por decisión entre 40–200%.
- 5. **Red de seguridad híbrida:** El LLM produce intención de alto nivel cada 0.5–2 s; un controlador PID rápido en C++/Blueprints opera a 100–200 Hz. Si el LLM falla o se ralentiza → activación de “hover seguro + retorno a casa”.

Razones por las que este enfoque sí se considera innovador

- La mayoría de los trabajos entre 2025 y 2026 siguen recurriendo a LLMs en la nube, hardware más potente o alimentaciones de estado sin tratamiento semántico.
- La combinación de *compresión semántica + autocorrección con gramática + aceleración especulativa + control híbrido* ejecutada en solo 8 GB de VRAM compartidos con una simulación UE es poco habitual, y potencialmente publicable como una aproximación de *control agente de dron en simulación de alta fidelidad con recursos limitados*.
- Si se logra una navegación estable y reactiva (evitación de obstáculos, búsqueda de objetivos, comportamiento dependiente de batería) con latencias por decisión inferiores a 500 ms, se trataría de una demostración sólida.

Se recomienda comenzar por implementar la *compresión semántica* y la *salida con gramática*, medir tasa de fallos y comparar con el bucle base. Posteriormente, incorporar speculative decoding como mejora incremental.

Un **Modelo de Lenguaje Pequeño (SLM)** es una versión ligera de un modelo de lenguaje tradicional, diseñada para operar de manera eficiente en entornos con recursos limitados, como teléfonos inteligentes, sistemas embebidos o computadoras de bajo consumo energético 1-5. Aquí hay una descripción más detallada de lo que son los SLMs:

- **Definición Operativa 6:**
 - Un SLM es un modelo de lenguaje (LM) que **puede instalarse en un dispositivo electrónico de consumo común** 4, 6.
 - Puede realizar inferencias con una **latencia suficientemente baja** para ser práctico al atender las solicitudes de un solo usuario en sistemas de agentes 5-8.
 - Un LLM se define como un LM que no es un SLM 6.
- **Tamaño y Escala** 4, 6, 9:
 - Mientras que los Modelos de Lenguaje Grandes (LLMs) tienen cientos de miles de millones, o incluso billones, de parámetros, los SLMs generalmente varían de **1 millón a 10 mil millones de parámetros** 4. A partir de 2025, se considerarían SLMs la mayoría de los modelos con menos de 10 mil millones de parámetros 6.
 - Es importante destacar que el término "pequeño" es relativo y se utiliza en comparación con los LLMs más grandes, ya que incluso un modelo de mil millones de parámetros no es "pequeño" por definición absoluta 9, 10.
- **Capacidades y Propósito** 4, 7, 11:
 - Los SLMs son suficientemente potentes para manejar las tareas de modelado de lenguaje de las aplicaciones de agentes 11, 12.
 - Mantienen capacidades básicas de Procesamiento de Lenguaje Natural (NLP) como generación de texto, resumen, traducción y respuesta a preguntas 4.
 - Se afirman como el futuro de la IA agéntica porque son inherentemente más adecuados operacionalmente y necesariamente más económicos para la mayoría de los usos de modelos de lenguaje en sistemas de agentes 11, 13.
- **Ventajas Clave** 5, 7, 14, 15:
 - **Menores requisitos computacionales:** Pueden ejecutarse en laptops de consumo, dispositivos de borde y teléfonos móviles 5, 8.
 - **Menor consumo de energía:** Modelos eficientes que reducen el uso de energía, haciéndolos más sostenibles 5, 8, 16.
 - **Inferencia más rápida:** Generan respuestas rápidamente, ideal para aplicaciones en tiempo real 5, 7, 8.
 - **IA en el dispositivo (On-Device AI):** No requieren conexión a internet ni servicios en la nube, lo que mejora la privacidad y la seguridad 5, 17.
 - **Despliegue más económico:** Menores costos de hardware y nube, lo que hace la IA más accesible 5, 14.
 - **Mayor flexibilidad y personalización:** Son más fáciles de ajustar para tareas específicas de dominio 5, 15.
- **Cómo se logran "pequeños"** 1, 9, 18:
 - **Destilación de conocimiento:** Entrenamiento de un modelo "estudiante" más pequeño utilizando el conocimiento transferido de un modelo "maestro" más grande 9, 18, 19.
 - **Poda (Pruning):** Eliminación de parámetros redundantes o menos importantes dentro de la arquitectura de la red neuronal 9, 20.
 - **Cuantización:** Reducción de la precisión de los valores numéricos utilizados en los cálculos (por ejemplo, convertir números de punto flotante a enteros) 9, 21.
- **Aplicaciones Comunes** 22:
 - Chatbots y asistentes virtuales.
 - Generación de código.
 - Traducción de idiomas.
 - Resumen y generación de contenido.
 - Aplicaciones en salud.
 - IoT y computación de borde.
 - Herramientas educativas.

En resumen, los SLMs son modelos eficientes y adaptables que buscan democratizar el acceso a la IA, permitiendo su despliegue en una gama más amplia de dispositivos y situaciones donde los LLMs serían inviables debido a sus altos requisitos de recursos 3, 4, 23, 24.

15.3 A3. SLM — Optimización y Desafíos

Nota de ubicación: notas de investigación sobre optimización y desafíos de los SLM. Material de referencia para [08-DECISIONES-SLM.md](#) §8.1. Reubicado desde [informe/SLM Optimización y Desafíos.md](#) el 2026-08-25.

Un **Modelo de Lenguaje Pequeño (SLM)** busca ser una versión eficiente y de bajo consumo de un modelo de lenguaje, diseñada para operar en entornos con recursos limitados 1, 2. La meta es mantener la precisión y/o adaptabilidad de los modelos de lenguaje grandes (LLMs) bajo ciertas restricciones, como hardware de entrenamiento o inferencia, disponibilidad de datos, ancho de banda o tiempo de generación 3.

Para reducir la cantidad de parámetros de un SLM sin perder la precisión de un LLM, se emplean varias técnicas clave:

- **Destilación de Conocimiento (Knowledge Distillation):**

- Esta técnica implica entrenar un modelo "estudiante" más pequeño para replicar el comportamiento de un modelo "maestro" más grande y complejo 4, 5.
- La destilación de conocimiento puede transferir las capacidades matizadas del LLM al SLM, permitiendo que el modelo pequeño imite las salidas del LLM en conjuntos de datos específicos de la tarea 5, 6.
- Por ejemplo, BabyLlama, un modelo compacto de 58 millones de parámetros, fue desarrollado utilizando un modelo Llama como "maestro", demostrando que la destilación puede superar el rendimiento de los modelos "maestros", especialmente en condiciones de datos limitados 5, 7, 8.
- Se puede mejorar la calidad de las respuestas de los modelos "estudiantes" mediante modificaciones en la función de pérdida de destilación 5. También es posible fusionar múltiples modelos de lenguaje como un solo "maestro" para destilar conocimiento en SLMs 9.
- Incluso se pueden destilar cadenas de razonamiento de un LLM más grande a un SLM, lo que ha demostrado mejorar las habilidades de razonamiento aritmético, matemático de varios pasos, simbólico y de sentido común en los modelos pequeños 10.
- La utilización de "razones" como supervisión adicional durante la destilación puede hacerla más eficiente en el uso de muestras, e incluso permitir que el modelo destilado supere a los LLMs en benchmarks comunes 10.

- **Poda (Pruning):**

- La poda es una técnica que reduce el número de parámetros del modelo para mejorar la eficiencia computacional y disminuir el uso de memoria, manteniendo al mismo tiempo los niveles de rendimiento 4, 11.
- Existen dos enfoques principales:
- **Poda no estructurada:** Elimina pesos individuales menos significativos, ofreciendo un control más granular para reducir el tamaño del modelo 11. Técnicas como SparseGPT pueden manejar modelos a gran escala, y la estrategia de poda $n:m$ (eliminar exactamente n pesos de cada m) busca equilibrar la flexibilidad de la poda con la eficiencia computacional para lograr aceleraciones significativas 11-13.
- **Poda estructurada:** Comprime los LLMs eliminando grupos de parámetros de manera estructurada, lo que facilita una implementación de hardware más eficiente 14. Esto incluye abordar la redundancia en la arquitectura Transformer y la escasez de neuronas, o la poda de capas 14, 15.
- **Cuantización (Quantization):**
- La cuantización reduce la precisión de los valores numéricos utilizados en los cálculos (por ejemplo, convertir números de punto flotante a enteros) para comprimir los LLMs con vastos recuentos de parámetros 4, 16.
- Métodos como GPTQ se centran en la cuantización de solo pesos por capa, minimizando el error de reconstrucción 16. Otros métodos cuantifican tanto los pesos como las activaciones 16.
- Técnicas como AWQ y ZeroQuant tienen en cuenta las activaciones para evaluar la importancia de los pesos, optimizando la cuantización de manera más efectiva 16.
- La Cuantización del Caché K/V (Key-Value Cache) se aplica específicamente para la inferencia eficiente de secuencias largas 16.
- SmoothQuant y SpinQuant abordan los valores atípicos en la distribución de activaciones, migrando la dificultad de cuantificación de las activaciones a los pesos o transformando los valores atípicos en un nuevo espacio 17.
- Los métodos de entrenamiento conscientes de la cuantización (QAT), como LLM-QAT y Edge-QAT, utilizan la destilación con modelos float16 para recuperar el error de cuantificación, demostrando un rendimiento sólido 17.

Otras técnicas y avances que contribuyen a la eficiencia y el mantenimiento de la precisión:

- **Arquitecturas Ligeras:** Diseños como MobileBERT (reducción de tamaño 4.3x y aceleración 5.5x sobre BERT), DistilBERT y TinyBERT 18 son optimizaciones de modelos encoder-only. Las arquitecturas decoder-only ligeras como BabyLLaMA, TinyLLaMA, MobilLLaMA y MobileLLM utilizan destilación de conocimiento, optimización de la sobrecarga de memoria, y esquemas de compartición de parámetros y embeddings 7.
- **Aproximaciones Eficientes de Autoatención:** Métodos como Reformer, mecanismos de atención lineal, Mamba y RWKV reducen la complejidad computacional de la autoatención de $O(N^2)$ a $O(N \log N)$ o $O(N)$ 19.
- **Búsqueda de Arquitectura Neural (NAS):** Métodos automatizados descubren las arquitecturas de modelos más eficientes para tareas y hardware específicos 20.
- **Entrenamiento de Precisión Mixta:** Técnicas como Automatic Mixed Precision (AMP), BFLOAT16 y FP8 mejoran la eficiencia del preentrenamiento al usar representaciones de baja precisión para la propagación hacia adelante y hacia atrás 21.

- **Ajuste Fino Eficiente en Parámetros (PEFT):** Técnicas como LoRA y Prompt Tuning actualizan solo un pequeño subconjunto de parámetros o añaden módulos ligeros, reduciendo los costos computacionales y preservando el conocimiento del modelo preentrenado 22-24.
- **Aumento de Datos:** El aumento de datos, a través de técnicas como AugGPT, Evol-Instruct y Reflection-tuning, incrementa la complejidad, diversidad y calidad de los datos de entrenamiento para SLMs, mejorando la generalización y el rendimiento en tareas específicas, especialmente cuando los datos son limitados 25, 26.
- **Potencia de Razonamiento:** Los SLMs ya poseen suficiente poder de razonamiento para una porción sustancial de las invocaciones de agentes. La capacidad, no el recuento de parámetros, es la restricción. Con el entrenamiento, el prompting y las técnicas de aumento agéntico modernos, los SLMs son viables y, comparativamente, más adecuados que los LLMs para sistemas agénticos modulares y escalables 27. Por ejemplo, el modelo Phi-2 (2.7 mil millones de parámetros) de Microsoft logra puntuaciones de razonamiento de sentido común y generación de código a la par de modelos de 30 mil millones de parámetros, mientras que funciona aproximadamente 15 veces más rápido 28. DeepSeek-R1-Distill-Qwen-7B supera a modelos propietarios grandes como Claude-3.5-Sonnet-1022 y GPT-4o-0513 29.

En resumen, los SLMs logran un equilibrio entre tamaño y rendimiento, empleando una combinación de técnicas de compresión y optimización, junto con arquitecturas de diseño inteligente y entrenamiento específico, para ofrecer capacidades similares a las de los LLMs en tareas especializadas 4, 27, 30.

SLMs y la Propensión a Alucinaciones

En cuanto a la propensión a alucinaciones, los Modelos de Lenguaje Grandes (LLMs) son conocidos por el problema de la "alucinación", que se define como la generación de contenido sin sentido o falso en relación con ciertas fuentes 31.

En el contexto de los SLMs:

- Un estudio utilizando **HallusionBench**, un benchmark para el razonamiento en modelos de visión-lenguaje, encontró que **los tamaños de modelo más grandes reducían las alucinaciones** 32. Esto sugiere que, en general, los modelos más pequeños podrían ser más propensos a generar contenido alucinatorio.
- El análisis del benchmark de alucinaciones AMBER también indicó que el tipo de alucinación varía a medida que cambia el recuento de parámetros en Minigpt-4 32.
- Las alucinaciones son un riesgo y una limitación que los SLMs comparten con los LLMs 33.
- La investigación futura necesita considerar no solo cómo cambia el total de alucinaciones en los SLMs, sino también cómo el tipo y la gravedad pueden verse influenciados por el tamaño del modelo 32.

Por lo tanto, existe evidencia que sugiere que los SLMs podrían ser más susceptibles a las alucinaciones debido a su menor tamaño, aunque este es un campo de investigación activo para comprender completamente la relación entre el tamaño del modelo y la naturaleza de las alucinaciones 32.

Nota de ubicación: notas de investigación sobre LoRA (Low-Rank Adaptation) aplicado a SLM. No hay evidencia en el código actual de que LoRA se haya implementado en el pipeline final; tratar como exploración de alternativas, no como decisión adoptada, salvo que `08-DECISIONES-SLM.md` documente lo contrario. Reubicado desde `informe/Optimización de Modelos de Lenguaje Pequeños mediante LoRA.md` el 2026-08-25.

La optimización de Modelos de Lenguaje Pequeños (SLM) con **LoRA (Low-Rank Adaptation)** es una estrategia altamente eficiente que forma parte de las técnicas de **Ajuste Fino Eficiente en Parámetros (PEFT)** 1, 2.

LoRA optimiza los modelos funcionando mediante una **descomposición de bajo rango**: actualiza solo un subconjunto muy pequeño de parámetros (o afina unas pocas capas específicas) mientras mantiene fijos la mayor parte de los parámetros del modelo preentrenado original 2, 3.

La aplicación de LoRA en SLMs aporta las siguientes ventajas y características fundamentales:

- **Eficiencia de recursos:** Al actualizar solo una fracción de la red, LoRA **reduce drásticamente los costos computacionales y los requisitos de memoria** asociados con el proceso de ajuste fino (fine-tuning), haciéndolo mucho más ligero y accesible 1-3.
- **Agilidad extrema:** El ajuste de un SLM utilizando LoRA requiere **solo unas pocas horas de procesamiento en GPU**. Esto permite a los desarrolladores un ciclo de iteración muy rápido para agregar nuevos comportamientos, corregir errores o especializar el modelo de la noche a la mañana, en lugar de esperar semanas 4.
- **Prevención del sobreajuste (Overfitting):** Dado que la mayor parte del modelo original permanece inalterada, LoRA ayuda a **preservar el conocimiento preentrenado del modelo**, reduce el riesgo de sobreajuste y mejora la flexibilidad 2.
- **Especialización de dominio:** Es el método ideal para adaptar un SLM general a **conjuntos de datos de dominios específicos o aplicaciones de nicho**. Por ejemplo, un modelo puede optimizarse de forma rápida con LoRA sobre documentos legales para crear un asistente de análisis de contratos, o sobre manuales técnicos para desarrollar una guía de resolución de problemas 5.
- **Variantes avanzadas y facilidad de uso:** Su implementación hoy en día es sencilla gracias a bibliotecas como peft de Hugging Face, que permiten configurar rápidamente los parámetros de la adaptación 3. Además, existen variantes populares empleadas en SLMs como **QLoRA** (que cuantiza el modelo para reducir aún más el consumo de recursos) y **DoRA**, que expanden la capacidad de ajustar modelos bajo restricciones de hardware 1, 4.

Según las fuentes, la generación de salidas estructuradas y la mejora en la eficiencia de la inferencia se logra principalmente a través de una técnica conocida como **decodificación restringida (constrained decoding)** 1, 2.

Generación de salidas estructuradas:

- La decodificación restringida interviene en el proceso de generación del modelo evaluando las reglas de una gramática o restricción dada y **enmascarando (ocultando) los tokens que son inválidos** en cada paso 2, 3.
- Al hacer esto, el modelo es guiado para que tome muestras únicamente de tokens válidos, lo que garantiza que la salida final se ajuste perfectamente a la estructura predefinida, siendo **JSON Schema** el estándar predominante en la industria para definir estos formatos 2, 4.
- Para lograr esto, se han desarrollado motores de gramática y marcos de trabajo optimizados como Guidance, Outlines, Llamacpp y XGrammar, los cuales traducen estas reglas para controlar las respuestas del modelo 5, 3.
- En el caso específico de los SLM integrados en sistemas de agentes autónomos, mantener formatos estrictos (como JSON, XML o código Python) es vital para comunicarse con otras herramientas 6. Las fuentes sugieren que los SLM pueden ser ajustados (fine-tuned) de forma económica para forzar una única decisión de formato, evitando así alucinaciones estructurales que rompan el código del sistema 6.

Mayor eficiencia en la inferencia: Aunque aplicar gramáticas o restricciones podría parecer un proceso que añade carga computacional, las implementaciones optimizadas en realidad **pueden acelerar el proceso de generación hasta en un 50%** en comparación con la generación sin restricciones 7. Esto se logra mediante varias optimizaciones clave:

- **Procesamiento en paralelo:** El cálculo de la máscara de tokens permitidos se ejecuta en paralelo con el paso hacia adelante (forward pass) del modelo de lenguaje 8.
- **Compilación simultánea:** La compilación inicial de la gramática requerida se realiza de manera concurrente con los cálculos de pre-llenado (pre-filling) del prompt inicial 8, 9.
- **Optimizaciones avanzadas:** Los sistemas emplean técnicas como el almacenamiento en caché de gramáticas y la decodificación especulativa basada en restricciones para reducir los tiempos de respuesta 8. Además, marcos como *Guidance* alcanzan una eficiencia sobresaliente al ser capaces de acelerar y saltarse directamente ciertos pasos de generación cuando la gramática los hace predecibles 10.